

PROJET DE FIN D'ÉTUDES

Filière : Développement Informatique

VALIQUO

*Conception et développement d'une plateforme
SaaS intelligente d'assistance réglementaire
destinée aux entrepreneurs marocains*

Présenté par :
Aïcha Soumaré

Encadré par :
M.Aid Daoud

Table des matières

Dédicaces.....	6
Remerciements	6
Résumé.....	7
Abstract	7
Introduction générale.....	8
Contexte général du projet	8
Problématique.....	8
Objectifs du projet.....	8
Méthodologie adoptée.....	8
Organisation du rapport.....	8
Chapitre 1 : Présentation et cadrage du projet.....	9
1.1 Présentation du projet	9
Titre du projet.....	9
Description générale.....	9
Contexte du projet	9
Public cible / utilisateurs concernés	9
1.2 Problématique.....	9
Problème constaté.....	9
Limites des solutions existantes	9
Besoin à satisfaire.....	10
1.3 Objectifs du projet.....	10
Objectif général	10
Objectifs spécifiques	10
Résultats attendus	10
1.4 Périmètre du projet.....	10
Fonctionnalités incluses	10
Fonctionnalités non incluses	11
Contraintes du projet	11
1.5 Méthodologie de gestion de projet adoptée	11
Présentation de la méthode choisie.....	11
Justification du choix.....	11
Étapes principales du projet	12
1.6 Planning prévisionnel	12
Découpage du projet en étapes.....	12
Planning de réalisation	13

Livrables prévus	13
Chapitre 2 : Analyse des besoins	14
2.1 Identification des acteurs	14
Acteur principal	14
Acteurs secondaires	14
Tableau récapitulatif des acteurs	14
2.2 Besoins fonctionnels	15
Authentification	15
Gestion des utilisateurs	15
Gestion des données principales	15
Gestion des traitements métier	15
Consultation, ajout, modification et suppression	16
Fonctionnalités liées à l'intelligence artificielle	16
Fonctionnalités liées à l'application mobile Flutter	16
2.3 Besoins non fonctionnels	16
Sécurité	16
Performance	16
Simplicité d'utilisation	17
Interface responsive	17
Compatibilité web et mobile	17
Fiabilité	17
Maintenabilité	17
2.4 Règles de gestion	17
Règles liées aux utilisateurs	17
Règles liées aux traitements	18
Règles liées aux statuts	18
Règles liées aux droits d'accès	18
2.5 Cahier des charges fonctionnel	18
Chapitre 3 : Conception UML et base de données	19
3.1 Introduction à la conception	19
Objectif de la conception	19
Passage de l'analyse vers la modélisation	19
3.2 Diagramme de cas d'utilisation	19
Présentation des acteurs	19
Présentation des cas d'utilisation	19
Description des principaux cas d'utilisation	20

3.3 Diagramme de classes	21
Présentation des classes principales	21
3.4 Schéma relationnel de la base de données.....	23
Présentation des tables.....	23
3.5 Diagramme de séquence 1.....	25
Scénario choisi	25
Description du scénario	25
Acteurs et participants impliqués :	25
Explication du déroulement.....	25
3.6 Diagramme de séquence 2.....	26
Scénario choisi	26
Description du scénario	26
Acteurs et participants impliqués :	26
Explication du déroulement.....	26
3.7 Diagramme de déploiement simple	27
Chapitre 4 : Architecture technique et technologies utilisées	29
4.1 Architecture générale de l'application	29
Architecture web	29
Architecture mobile	29
4.2 Technologies côté client.....	29
4.3 Technologies côté serveur.....	30
4.4 Base de données	31
4.5 Application mobile Flutter.....	31
4.6 Intégration de l'intelligence artificielle	32
Type d'intégration choisi	32
Rôle de l'IA dans le projet	32
4.7 Outils de développement et DevOps	32
Chapitre 5 : Réalisation de l'application web.....	34
5.1 Présentation générale de l'application web	34
5.2 Authentification et gestion des utilisateurs	35
5.3 Tableau de bord	35
5.4 Gestion des principales fonctionnalités	36
5.5 Présentation des interfaces.....	37
5.6 Difficultés rencontrées et solutions adoptées	40
Chapitre 6 : Réalisation de l'application mobile Flutter.....	42
6.1 Objectif de l'application mobile	42

Rôle de la partie mobile dans le projet	42
Public cible	42
Fonctionnalités mobiles prévues	42
6.2 Interfaces principales	42
6.3 Communication avec le backend Laravel	43
API utilisée	43
6.4 Captures d'écran de l'application mobile	44
Chapitre 7 : Intégration de l'intelligence artificielle	48
7.1 Objectif de l'intégration IA	48
7.2 Solution IA choisie	48
7.3 Fonctionnement technique	49
7.4 Exemples d'utilisation	50
Chapitre 8 : Tests, validation et livraison	52
8.1 Stratégie de test	52
Conclusion générale	57
Bilan du projet	57
Difficultés rencontrées	57
Limites du travail	57
Perspectives d'amélioration	58
Bibliographie / Webographie	58

Dédicaces

Alhamdulillah, par la grâce d'Allah, j'ai pu mener ce travail à son terme.

Je dédie ce travail à toutes celles et ceux qui ont cru en moi et m'ont soutenue tout au long de ce parcours.

À mes parents, dont l'amour inconditionnel, les sacrifices et les précieux conseils ont toujours été ma plus grande force et source d'inspiration.

À mes frères et sœurs, pour leur soutien indéfectible et leur complicité. Vos encouragements ont été un moteur essentiel dans l'aboutissement de ce projet.

À mes amis, dont la présence, l'écoute et la bonne humeur ont illuminé les moments les plus exigeants.

Enfin, je dédie ce travail à tous ceux qui m'ont appris la persévérance, l'importance de croire en mes rêves et la force de ne jamais abandonner, même face aux défis les plus ardues.

Que chacun trouve ici l'expression de ma profonde gratitude et de mon affection.

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce projet. Leur soutien, leurs conseils et leur accompagnement ont été précieux pour mener à bien ce travail.

Tout d'abord, je souhaite remercier mes encadrants pédagogiques pour leur accompagnement bienveillant, leur disponibilité et leurs conseils avisés tout au long de ce projet.

Ma reconnaissance s'étend également à l'ensemble de l'équipe pédagogique d'**Omnia School**, dont la qualité de l'enseignement et l'expertise m'ont permis d'acquérir des compétences essentielles. Un merci particulier aux intervenants professionnels qui ont partagé avec générosité leur savoir et leur expérience, enrichissant ainsi mon parcours.

Enfin, une pensée toute particulière pour ma famille, dont l'amour, le soutien inconditionnel et les encouragements m'ont accompagnée tout au long de ce chemin. Leur présence a été ma plus grande force et une motivation précieuse pour avancer.

Que toutes les personnes ayant contribué à ce projet trouvent ici l'expression de ma sincère reconnaissance.



Résumé

Valiquo est une plateforme SaaS intelligente destinée à accompagner les entrepreneurs marocains dans la compréhension du cadre réglementaire et juridique applicable à la création et à la gestion d'une entreprise au Maroc. Face à la complexité des procédures administratives, à la dispersion des informations officielles entre plusieurs organismes (OMPIC, DGI, CRI, CNSS) et au coût élevé des consultants spécialisés, Valiquo propose une alternative accessible et fiable grâce à l'intelligence artificielle.

La plateforme s'appuie sur une architecture découplée associant un frontend React (TypeScript), un backend Laravel exposant une API REST sécurisée (authentification via Laravel Breeze, Sanctum et gestion des rôles avec Spatie Permission), une base de données MySQL, ainsi qu'une intelligence artificielle externe (API Google Gemini) exploitée selon une approche de type RAG (Retrieval-Augmented Generation), permettant de générer des réponses contextualisées à partir de documents réglementaires officiels marocains.

Le projet, initialement conçu comme une plateforme généraliste de validation d'idées entrepreneuriales, a été recentré sur un module réglementaire (V1) suite aux retours des encadrants pédagogiques, dans une logique d'évolution progressive vers des modules complémentaires (financement, analyse sectorielle, données macroéconomiques). Une application mobile complémentaire a également été développée avec Flutter.

Ce travail illustre une démarche complète de conception et de réalisation d'un projet informatique transversal, intégrant analyse des besoins, modélisation UML, développement web et mobile, ainsi qu'intégration d'une intelligence artificielle.

Abstract

Valiquo is an intelligent SaaS platform designed to assist Moroccan entrepreneurs in understanding the regulatory and legal framework applicable to business creation and management in Morocco. Faced with the complexity of administrative procedures, the fragmentation of official information across multiple institutions (OMPIC, DGI, CRI, CNSS), and the high cost of specialized consultants, Valiquo offers an accessible and reliable alternative powered by artificial intelligence.

The platform relies on a decoupled architecture combining a React (TypeScript) frontend, a Laravel backend exposing a secure REST API (authentication via Laravel Breeze, Sanctum, and role management with Spatie Permission), a MySQL database, and an external artificial intelligence service (Google Gemini API) used through a Retrieval-Augmented Generation (RAG) approach, enabling contextualized answers generated from official Moroccan regulatory documents. Initially conceived as a general-purpose business idea validation platform, the project was refocused on a regulatory module (V1) following feedback from academic supervisors, as part of a progressive evolution toward complementary modules (financing, sector analysis, macroeconomic data). A companion mobile application was also developed using Flutter.

This work illustrates a complete approach to designing and implementing a cross-disciplinary computer science project, encompassing needs analysis, UML modeling, web and mobile development, and the integration of artificial intelligence.



Introduction générale

Contexte général du projet

Dans un contexte économique en pleine mutation, le Maroc connaît une dynamique entrepreneuriale croissante, portée par une jeunesse ambitieuse et des politiques publiques favorables à la création d'entreprise. Cependant, malgré cet élan, de nombreux porteurs de projets se heurtent à un obstacle majeur : la complexité du cadre réglementaire et juridique marocain. Entre les procédures administratives dispersées, les textes de loi difficiles à interpréter et le coût élevé des consultants spécialisés, l'accès à une information fiable et contextualisée reste un défi pour la majorité des entrepreneurs, notamment les primo-créateurs.

Problématique

Comment permettre à un entrepreneur marocain d'accéder rapidement à une information réglementaire fiable, structurée et adaptée à son projet, sans avoir recours systématiquement à un expert juridique coûteux ?

Objectifs du projet

Ce projet vise à concevoir et développer **Valiquo**, une plateforme SaaS intelligente qui met à disposition des entrepreneurs marocains un assistant IA spécialisé en droit des affaires marocain. Grâce à la technique du RAG (Retrieval-Augmented Generation), l'application analyse les questions des utilisateurs et génère des réponses précises basées sur les textes officiels marocains en vigueur Code Général des Impôts, loi sur la SARL, procédures OMPIC, statut auto-entrepreneur, entre autres.

Méthodologie adoptée

Le projet a été réalisé selon un **cycle en cascade**, permettant de structurer le développement en phases successives et bien délimitées : analyse des besoins, conception, développement web, développement mobile, intégration de l'IA, puis tests et validation. Cette approche a facilité le suivi de l'avancement et la communication avec les encadrants pédagogiques.

Organisation du rapport

Ce rapport est structuré en huit chapitres. Le premier présente le cadrage général du projet. Le second détaille l'analyse des besoins fonctionnels et non fonctionnels. Le troisième couvre la conception UML et la modélisation de la base de données. Le quatrième décrit l'architecture technique et les technologies utilisées. Les chapitres cinq et six présentent respectivement la réalisation de l'application web et de l'application mobile. Le chapitre sept détaille l'intégration de l'intelligence artificielle. Enfin, le chapitre huit présente les tests réalisés et la validation du projet.



Chapitre 1 : Présentation et cadrage du projet

1.1 Présentation du projet

Titre du projet
Valiquo

Description générale

Valiquo est une plateforme SaaS intelligente destinée aux entrepreneurs marocains, qui permet d'obtenir des réponses précises et fiables aux questions réglementaires et juridiques liées à la création et à la gestion d'une entreprise au Maroc. La plateforme s'appuie sur un système RAG (Retrieval-Augmented Generation) alimenté par des textes officiels marocains, garantissant des réponses contextualisées et à jour.

Contexte du projet

Le projet s'inscrit dans le cadre du Projet de Fin d'Études de deuxième année en développement informatique à Omnia School. Il a été initié en réponse à un besoin réel identifié sur le marché marocain : la difficulté d'accès à une information réglementaire fiable pour les entrepreneurs, notamment les primo-créateurs.

Public cible / utilisateurs concernés

La plateforme s'adresse principalement aux entrepreneurs marocains, qu'ils soient en phase de création d'entreprise ou déjà en activité, cherchant à clarifier leurs obligations légales et fiscales. Un second type d'utilisateur, l'administrateur, gère la plateforme et alimente la base de connaissances de l'IA.

1.2 Problématique

Problème constaté

Au Maroc, de nombreux entrepreneurs, en particulier les primo-créateurs, rencontrent d'importantes difficultés à comprendre et appliquer le cadre réglementaire lié à la création et à la gestion d'une entreprise. Les informations officielles, fiscalité, statuts juridiques, procédures administratives sont dispersées entre plusieurs organismes (OMPIC, DGI, CRI, CNSS) et rédigées dans un langage juridique souvent difficile d'accès pour un non-spécialiste.

Limites des solutions existantes

Plusieurs solutions existent actuellement mais présentent des limites significatives :

- **Les consultants juridiques et experts-comptables** offrent un accompagnement personnalisé mais à un coût élevé (500 à 2000 MAD de l'heure), inaccessible à de nombreux porteurs de projets
- **Les moteurs de recherche généralistes** renvoient des informations parfois obsolètes, contradictoires ou non vérifiées
- Les assistants conversationnels généralistes ne disposent pas nécessairement d'une base documentaire spécialisée sur la réglementation marocaine, ce qui peut limiter la pertinence de certaines réponses dans ce contexte spécifique.



- **Les sites institutionnels officiels** sont riches en information mais peu intuitifs et ne permettent pas une recherche contextualisée selon le projet de l'utilisateur

Besoin à satisfaire

Il existe donc un besoin réel pour un outil accessible, gratuit ou à faible coût, capable de fournir des réponses fiables, précises et actualisées sur la réglementation marocaine, adaptées à la situation spécifique de chaque entrepreneur, sans nécessiter de connaissances juridiques préalables.

1.3 Objectifs du projet

Objectif général

Concevoir et développer une plateforme web et mobile intégrant une intelligence artificielle spécialisée, capable d'assister les entrepreneurs marocains dans la compréhension du cadre réglementaire applicable à leur projet, en s'appuyant sur des sources officielles fiables.

Objectifs spécifiques

- Développer une application web permettant aux utilisateurs de poser des questions réglementaires et d'obtenir des réponses structurées
- Mettre en place un système d'authentification sécurisé avec gestion des rôles (entrepreneur / administrateur)
- Intégrer un système RAG capable d'analyser des documents officiels marocains et de générer des réponses contextualisées
- Développer une application mobile Flutter permettant un accès simplifié aux fonctionnalités principales
- Assurer la traçabilité des consultations via un historique consultable par l'utilisateur
- Concevoir une architecture technique évolutive, capable d'accueillir de futurs modules (financement, données sectorielles)

Résultats attendus

À l'issue du projet, Valiquo doit permettre à un entrepreneur de soumettre une question réglementaire et d'obtenir, en quelques minutes, une réponse fiable basée sur les textes officiels marocains en vigueur, accompagnée d'un plan d'action concret. La plateforme doit être fonctionnelle sur web et mobile, sécurisée, et démontrer une intégration cohérente entre les différents modules techniques (backend, frontend, IA, mobile).

1.4 Périmètre du projet

Fonctionnalités incluses

- Authentification et gestion des comptes utilisateurs (inscription, connexion, gestion des rôles)
- Soumission de questions réglementaires via un formulaire structuré
- Génération de réponses via le système RAG basé sur des documents officiels marocains (fiscalité, création d'entreprise, droit des affaires)



- Interface de chat avec le Coach IA pour un accompagnement conversationnel
- Tableau de bord personnel avec historique des consultations
- Recherche et filtrage des consultations passées par thématique
- Interface d'administration pour la gestion des utilisateurs et des documents alimentant le RAG
- Application mobile Flutter offrant un accès simplifié aux fonctionnalités principales (consultation, chat, historique)

Fonctionnalités non incluses

- Le module financement (Maroc PME, Innov Invest, CCG) prévu pour une version ultérieure (V2)
- Le module d'analyse sectorielle et d'étude de marché prévu pour une version ultérieure (V3)
- L'intégration de données macroéconomiques officielles (HCP, Bank Al-Maghrib) prévue pour une version ultérieure (V4)
- Le paiement en ligne et la gestion des abonnements payants
- La génération automatique de documents juridiques personnalisés (statuts, contrats)

Contraintes du projet

- **Contrainte temporelle** : le projet devait être développé et présenté dans un délai global imparti par le calendrier académique, ce qui a conduit à recentrer le périmètre fonctionnel sur le module réglementaire (V1)
- **Contrainte budgétaire** : utilisation exclusive d'outils et de services gratuits ou à coût minimal (Gemini API, hébergement Vercel, base de données MySQL)
- **Contrainte technique** : nécessité d'assurer une communication fiable entre un frontend React, un backend Laravel et un service IA externe
- **Contrainte de données** : disponibilité limitée des documents réglementaires, ayant nécessité une collecte autonome de sources officielles publiques.

1.5 Méthodologie de gestion de projet adoptée

Présentation de la méthode choisie

Le projet Valiquo a été réalisé selon le **cycle en cascade** (Waterfall), une méthode structurée où chaque phase du développement suit logiquement la précédente : analyse des besoins, conception, développement, tests, puis livraison.

Justification du choix

Le cycle en cascade a été retenu pour plusieurs raisons. D'une part, le projet, bien que transversal (web, mobile, IA), reste un projet individuel avec un périmètre fonctionnel clairement défini dès le départ, ce qui correspond bien aux caractéristiques d'un cycle en cascade adapté aux projets simples et bien planifiés. D'autre part, le délai imparti pour la réalisation du projet a nécessité une planification linéaire et rigoureuse des étapes, sans possibilité de réaliser de nombreuses itérations comme le permettrait une méthode agile.

Le choix du périmètre fonctionnel ayant lui-même évolué en cours de projet recentrage de la plateforme sur le module réglementaire suite aux retours des encadrants pédagogiques une



phase de cadrage a été reconduite avant de poursuivre les phases suivantes du cycle, sans pour autant remettre en cause la structure générale en cascade.

Étapes principales du projet

1. **Analyse des besoins** : identification des acteurs, fonctionnalités et contraintes
2. **Conception**:modélisation UML, schéma de base de données
3. **Développement de l'application web** : frontend React et backend Laravel
4. **Intégration de l'intelligence artificielle** mise en place du système RAG
5. **Développement de l'application mobile**: Flutter
6. **Tests et validation** : vérification fonctionnelle et technique
7. **Livraison**: déploiement et documentation finale

1.6 Planning prévisionnel

Découpage du projet en étapes

Le projet a été découpé en plusieurs phases successives, conformément au cycle en cascade adopté :

Phase	Description
Cadrage et analyse	Définition de l'idée, identification des besoins, recentrage du périmètre sur le module réglementaire
Conception	Modélisation UML, conception de la base de données
Développement frontend	Conception et développement de l'interface web (React)
Développement backend	Mise en place de l'API Laravel, authentification, base de données
Intégration IA	Collecte des documents réglementaires, intégration du système RAG
Développement mobile	Développement de l'application Flutter
Tests et corrections	Vérification du bon fonctionnement de l'ensemble des modules
Finalisation et livraison	Déploiement, rédaction du rapport, préparation de la soutenance



Planning de réalisation

Semaine	Activités
Semaines 1-4	Cadrage initial, conception de la plateforme, développement frontend (landing, scanner, dashboard)
Semaine 5-6	Recentrage du périmètre sur le module réglementaire suite aux retours pédagogiques et Refonte du frontend, collecte des documents officiels marocains
Semaine 7	Développement backend Laravel, authentification, intégration du RAG
Semaine 8	Développement de l'application mobile, tests, finalisation et soutenance

Livrables prévus

- Application web fonctionnelle (frontend + backend)
- Application mobile Flutter
- Base de données MySQL avec script de sauvegarde
- Module IA intégré (RAG)
- Code source organisé et versionné sur GitHub
- Rapport de PFE complet
- Guide d'installation et README
- Support de présentation pour la soutenance.



Chapitre 2 : Analyse des besoins

2.1 Identification des acteurs

Acteur principal

Entrepreneur

L'entrepreneur constitue l'acteur principal du système. Il utilise la plateforme afin d'obtenir des informations réglementaires et administratives relatives à la création et à la gestion d'une entreprise au Maroc.

Ses principales actions sont :

- Créer un compte.
- Se connecter à la plateforme.
- Poser des questions réglementaires.
- Interagir avec le Coach IA.
- Consulter l'historique de ses consultations.
- Gérer son profil.

Acteurs secondaires

Administrateur

L'administrateur assure la gestion et le bon fonctionnement de la plateforme.

Ses principales responsabilités sont :

- Gérer les utilisateurs.
- Gérer les documents réglementaires utilisés par le système IA.
- Consulter les statistiques d'utilisation.
- Contrôler le contenu de la base documentaire.

Service d'intelligence artificielle (Gemini)

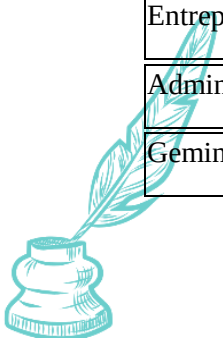
Le service IA constitue un acteur externe au système.

Il intervient pour :

- analyser les questions soumises ;
- exploiter les documents réglementaires ;
- générer des réponses contextualisées.

Tableau récapitulatif des acteurs

Acteur	Rôle
Entrepreneur	Utiliser la plateforme et obtenir des réponses réglementaires
Administrateur	Administrer les utilisateurs et la base documentaire
Gemini API	Générer les réponses basées sur les documents officiels



2.2 Besoins fonctionnels

Authentification

Le système doit permettre :

- l'inscription d'un utilisateur ;
- la connexion ;
- la déconnexion ;
- la modification du mot de passe ;
- la gestion des sessions.

Gestion des utilisateurs

Le système doit permettre :

Entrepreneur

- consulter son profil ;
- modifier ses informations personnelles ;
- consulter son historique.

Administrateur

- consulter la liste des utilisateurs ;
- activer ou désactiver un compte ;
- consulter les statistiques.

Gestion des données principales

Le système doit permettre :

- l'enregistrement des conversations ;
- la sauvegarde des consultations ;
- la gestion des documents réglementaires ;
- l'organisation de l'historique.

Gestion des traitements métier

Le système doit permettre :

- l'analyse des questions réglementaires ;
- la transmission des documents au moteur IA ;
- la génération de réponses contextualisées ;
- l'enregistrement des résultats.



Consultation, ajout, modification et suppression

Fonction	Entrepreneur	Administrateur
Consulter profil	Oui	Oui
Modifier profil	Oui	Oui
Consulter historique	Oui	Oui
Ajouter document	Non	Oui
Modifier document	Non	Oui
Supprimer document	Non	Oui

Fonctionnalités liées à l'intelligence artificielle

Le système doit permettre :

- la soumission de questions réglementaires ;
- l'analyse des documents réglementaires ;
- la génération de réponses contextualisées ;
- la conservation des conversations ;
- l'affichage des réponses dans une interface de chat.

Fonctionnalités liées à l'application mobile Flutter

L'application mobile doit permettre :

- l'authentification ;
- la consultation des conversations ;
- l'accès au Coach IA ;
- la consultation de l'historique ;
- la gestion du profil utilisateur.

2.3 Besoins non fonctionnels

Sécurité

Le système doit garantir :

- le hachage des mots de passe ;
- la protection des données utilisateurs ;
- le contrôle des accès ;
- la validation des données saisies.

Performance

Le système doit fournir des temps de réponse acceptables lors :



- des requêtes utilisateur ;
- des appels à l'API Gemini ;
- du chargement des interfaces.

Simplicité d'utilisation

L'interface doit être :

- intuitive ;
- claire ;
- accessible à des utilisateurs non techniques.

Interface responsive

L'application web doit s'adapter :

- aux ordinateurs ;
- aux tablettes ;
- aux smartphones.

Compatibilité web et mobile

Le système doit fonctionner :

- sur navigateur web moderne ;
- sur Android via Flutter.

Fiabilité

Le système doit assurer :

- l'intégrité des données ;
- la disponibilité des services ;
- la cohérence des réponses générées.

Maintenabilité

L'architecture doit permettre :

- l'ajout de nouveaux modules ;
- la mise à jour des documents réglementaires ;
- l'évolution des fonctionnalités.

2.4 Règles de gestion

Règles liées aux utilisateurs

RG01 : Un utilisateur doit être authentifié pour accéder aux fonctionnalités privées.

RG02 : Chaque utilisateur possède un rôle unique.



RG03 : L'adresse email doit être unique.

Règles liées aux traitements

RG04 : Toute question réglementaire doit être enregistrée dans l'historique.

RG05 : Toute réponse générée doit être associée à son utilisateur.

RG06 : Les documents utilisés doivent provenir de sources officielles.

Règles liées aux statuts

RG07 : Un utilisateur peut être actif ou inactif.

RG08 : Un document réglementaire peut être actif ou archivé.

Règles liées aux droits d'accès

RG09 : Seul l'administrateur peut gérer les utilisateurs.

RG10 : Seul l'administrateur peut ajouter ou supprimer des documents réglementaires.

RG11 : Un entrepreneur ne peut consulter que ses propres données.

2.5 Cahier des charges fonctionnel

ID	Fonctionnalité	Description	Priorité
F01	Inscription	Création d'un compte utilisateur	Haute
F02	Connexion	Authentification utilisateur	Haute
F03	Gestion du profil	Consultation et modification du profil	Moyenne
F04	Coach IA	Interaction avec l'assistant réglementaire	Haute
F05	Historique	Consultation des anciennes conversations	Haute
F06	Gestion des utilisateurs	Administration des comptes	Haute
F07	Gestion documentaire	Ajout et mise à jour des documents réglementaires	Haute
F08	Statistiques	Tableau de bord utilisateur	Moyenne
F09	Application mobile	Accès mobile aux fonctionnalités principales	Moyenne
F10	Recherche dans l'historique	Filtrage et recherche des consultations	Faible



Chapitre 3 : Conception UML et base de données

3.1 Introduction à la conception

Objectif de la conception

Cette phase de conception vise à traduire les besoins fonctionnels et non fonctionnels identifiés au chapitre précédent en une représentation structurée et modélisée, à l'aide du langage UML (Unified Modeling Language). Elle permet de formaliser le comportement attendu du système, les entités qui le composent, ainsi que les interactions entre les différents acteurs et composants techniques.

Passage de l'analyse vers la modélisation

L'analyse des besoins a permis d'identifier les acteurs (Entrepreneur, Administrateur), le service externe Gemini API, ainsi que l'ensemble des fonctionnalités attendues. La phase de conception consiste désormais à organiser ces éléments sous forme de diagrammes UML : diagramme de cas d'utilisation, diagramme de classes, diagramme de séquence et diagramme de déploiement afin de préparer une implémentation technique cohérente et structurée.

3.2 Diagramme de cas d'utilisation

Présentation des acteurs

Le système Valiquo implique deux acteurs humains principaux :

- **Entrepreneur** : utilisateur standard de la plateforme
- **Administrateur** : gestionnaire de la plateforme et de la base documentaire

Le **service Gemini API** est représenté comme un système externe consommé par l'application, et non comme un acteur humain. Il intervient en relation avec le cas d'utilisation "Générer une réponse réglementaire".

Présentation des cas d'utilisation

Pour l'Entrepreneur :

- S'inscrire
- Se connecter
- Poser une question réglementaire
- Consulter une réponse
- Discuter avec le Coach IA
- Consulter l'historique
- Rechercher/filtrer dans l'historique
- Gérer son profil
- Se déconnecter

Pour l'Administrateur :



- Se connecter
- Gérer les utilisateurs (consulter, activer/désactiver)
- Ajouter un document réglementaire
- Modifier/mettre à jour un document
- Supprimer un document
- Consulter les statistiques

Relation avec le système externe :

- Le cas d'utilisation "Générer une réponse réglementaire" établit une relation avec le service Gemini API (flèche en pointillé <<include>> ou liaison vers système externe selon la convention adoptée)

Description des principaux cas d'utilisation

Cas d'utilisation	Acteur	Description
Poser une question réglementaire	Entrepreneur	L'utilisateur soumet une question via le formulaire scanner ; le système transmet la demande au service IA et enregistre la réponse
Gérer les documents réglementaires	Administrateur	L'administrateur ajoute, modifie ou supprime les documents PDF officiels qui alimentent la base de connaissances du système
Discuter avec le Coach IA	Entrepreneur	L'utilisateur engage une conversation libre avec l'assistant, qui répond en s'appuyant sur les documents disponibles



Role

- id (PK)
- nom (entrepreneur / admin)

Consultation

- id (PK)
- user_id (FK)
- question
- reponse
- thematique
- ville
- date_creation
- statut

Conversation

- id (PK)
- user_id (FK)
- titre
- date_creation

Message

- id (PK)
- conversation_id (FK)
- contenu
- expediteur (user / coach)
- date_envoi

Document

- id (PK)
- titre
- fichier_pdf
- categorie
- date_ajout
- admin_id (FK)

Relations entre les classes

- **User (1) — (1) Role** : un utilisateur possède un rôle
- **User (1) — (N) Consultation** : un utilisateur peut soumettre plusieurs consultations
- **User (1) — (N) Conversation** : un utilisateur peut démarrer plusieurs conversations
- **Conversation (1) — (N) Message** : une conversation contient plusieurs messages
- **User/Admin (1) — (N) Document** : un administrateur peut ajouter plusieurs documents

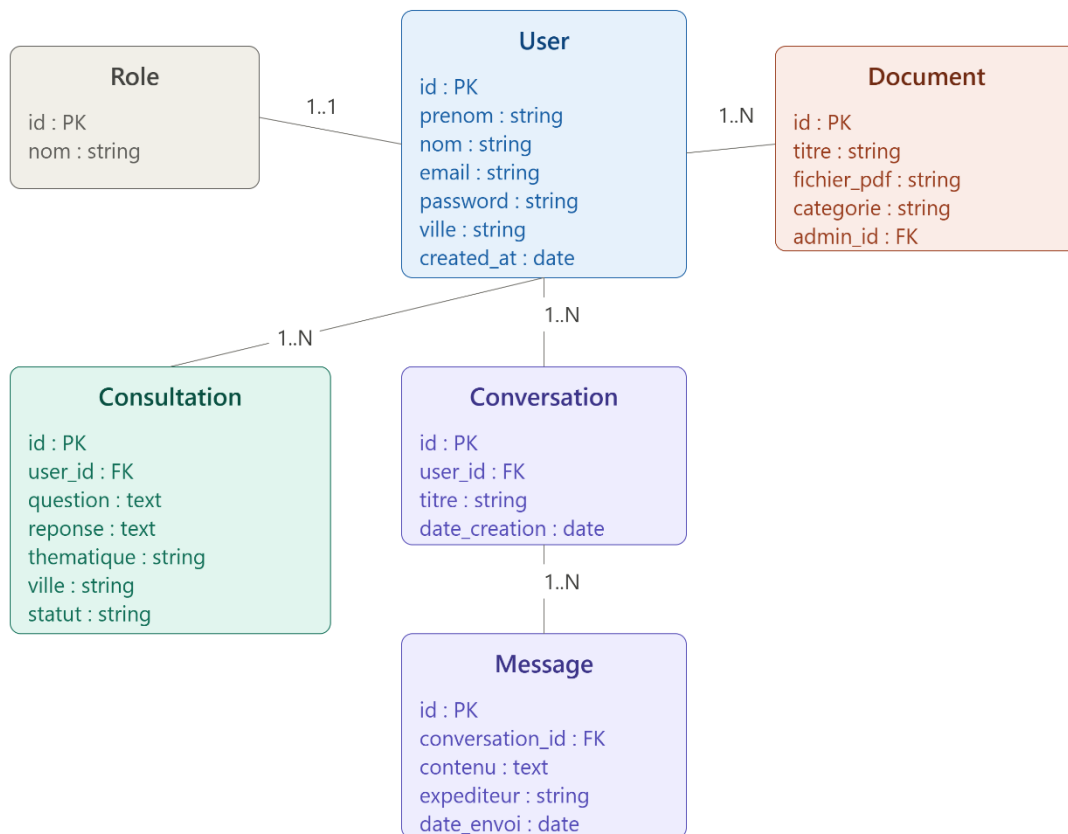
Explication des associations importantes



L'association entre **User** et **Consultation** est fondamentale car elle garantit la traçabilité chaque consultation est rattachée à un utilisateur précis, conformément à la règle de gestion RG05 définie au chapitre précédent (RG05 : toute réponse générée doit être associée à son utilisateur).

L'association entre **Conversation** et **Message** permet de structurer l'historique du Coach IA sous forme de fils de discussion distincts, plutôt qu'un flux unique de messages, ce qui facilite la navigation dans l'historique (fonctionnalité F05 du cahier des charges).

La relation entre **Document** et l'administrateur reflète la règle RG10 seul l'administrateur peut ajouter ou supprimer des documents réglementaires.



3.4 Schéma relationnel de la base de données

Présentation des tables

Le schéma relationnel de Valiquo comprend les tables suivantes :

Table	Description
users	Stocke les informations des utilisateurs
roles	Stocke les rôles disponibles (entrepreneur, admin)

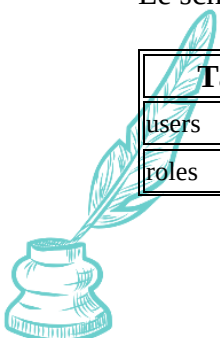


Table	Description
model_has_roles	Table pivot Spatie reliant users et roles
consultations	Stocke les questions réglementaires et leurs réponses
conversations	Stocke les fils de discussion avec le Coach IA
messages	Stocke les messages individuels d'une conversation
documents	Stocke les métadonnées des documents PDF réglementaires

Clés primaires

- users.id
- roles.id
- consultations.id
- conversations.id
- messages.id
- documents.id

Clés étrangères

- consultations.user_id → users.id
- conversations.user_id → users.id
- messages.conversation_id → conversations.id
- documents.admin_id → users.id
- model_has_roles.user_id → users.id
- model_has_roles.role_id → roles.id

Relations entre les tables

- users (1) — (N) consultations
- users (1) — (N) conversations
- conversations (1) — (N) messages
- users (1) — (N) documents (*côté administrateur*)
- users (N) — (N) roles *via* model_has_roles

Justification de la structure de la base de données

La séparation entre consultations et conversations/messages reflète deux usages distincts de l'IA dans Valiquo : les **consultations** correspondent à des questions ponctuelles et structurées soumises via le scanner (avec thématique et ville), tandis que les **conversations** représentent des échanges libres et continus avec le Coach IA, nécessitant une structure en fils de messages pour conserver le contexte conversationnel.

L'utilisation de **Spatie Laravel Permission** justifie la présence de la table pivot model_has_roles, qui permet une gestion flexible et extensible des rôles, plutôt qu'un simple champ role dans la table users. Cette approche facilite l'ajout de nouveaux rôles dans les versions futures de la plateforme (par exemple un rôle "modérateur" ou "partenaire").



La table documents est isolée afin de centraliser la gestion de la base de connaissances du système RAG, indépendamment des données utilisateurs, ce qui respecte le principe de séparation des responsabilités.

3.5 Diagramme de séquence 1

Scénario choisi

Soumission d'une question réglementaire par l'entrepreneur et génération de la réponse via le service Gemini API.

Description du scénario

Ce scénario illustre l'interaction principale du système : un entrepreneur connecté soumet une question réglementaire via le formulaire scanner, et le système traite cette demande en faisant appel au service IA externe pour générer une réponse contextualisée, avant de l'enregistrer et de l'afficher à l'utilisateur.

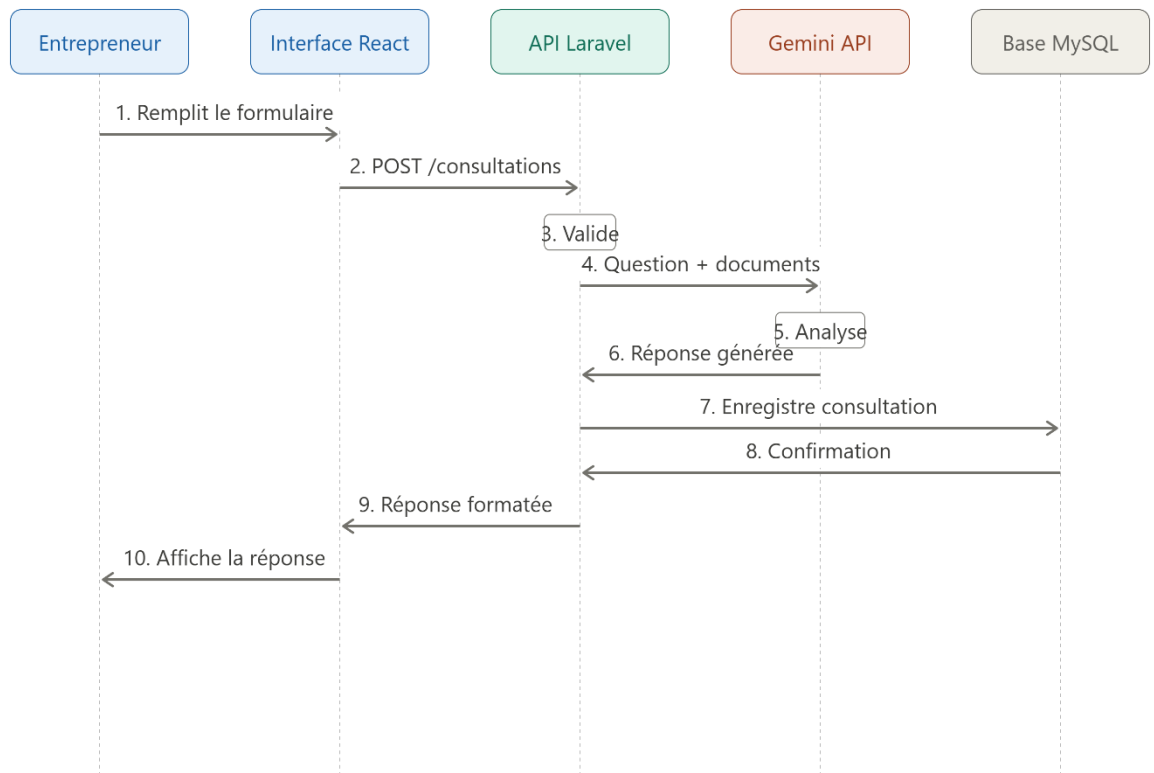
Acteurs et participants impliqués :

- Entrepreneur (acteur)
- Interface React (frontend)
- API Laravel (backend)
- Base de données MySQL
- Service Gemini API (système externe)

Explication du déroulement

1. L'**Entrepreneur** remplit le formulaire de question réglementaire (question, thématique, ville) sur l'**Interface React**
2. L'**Interface React** envoie une requête HTTP POST à l'**API Laravel** (/api/consultations)
3. L'**API Laravel** valide les données reçues
4. L'**API Laravel** transmet la question, accompagnée des documents réglementaires pertinents, au **service Gemini API**
5. Le **service Gemini API** analyse la question et les documents fournis, puis génère une réponse contextualisée
6. Le **service Gemini API** retourne la réponse générée à l'**API Laravel**
7. L'**API Laravel** enregistre la consultation (question + réponse) dans la **Base de données MySQL**
8. La **Base de données MySQL** confirme l'enregistrement
9. L'**API Laravel** retourne la réponse formatée à l'**Interface React**
10. L'**Interface React** affiche la réponse structurée à l'**Entrepreneur**





3.6 Diagramme de séquence 2

Scénario choisi

Ajout d'un document réglementaire par l'administrateur pour alimenter la base de connaissances du système RAG.

Description du scénario

Ce scénario illustre comment l'administrateur enrichit la base documentaire utilisée par le système IA. Il met en évidence le rôle critique de l'administrateur dans la qualité des réponses générées par Valiquo, conformément à la règle de gestion RG10 (seul l'administrateur peut ajouter ou supprimer des documents réglementaires).

Acteurs et participants impliqués :

- Administrateur (acteur)
- Interface React (frontend espace admin)
- API Laravel (backend)
- Base de données MySQL

Explication du déroulement

1. L'**Administrateur** se connecte et accède à l'interface de gestion documentaire



2. L'**Administrateur** sélectionne un fichier PDF officiel et renseigne ses métadonnées (titre, catégorie)
3. L'**Interface React** envoie une requête HTTP POST à l'**API Laravel** (/api/documents) avec le fichier et ses métadonnées
4. L'**API Laravel** vérifie que l'utilisateur possède bien le rôle administrateur (middleware Spatie)
5. L'**API Laravel** valide le format et la taille du fichier
6. L'**API Laravel** enregistre les métadonnées du document dans la **Base de données MySQL**
7. La **Base de données MySQL** confirme l'enregistrement
8. L'**API Laravel** retourne une confirmation de succès à l'**Interface React**
9. L'**Interface React** affiche un message de confirmation à l'**Administrateur** et met à jour la liste des documents.

3.7 Diagramme de déploiement simple

Client web

Le client web correspond à l'application **React (TypeScript)**, hébergée sur **Vercel**. Il s'exécute dans le navigateur de l'utilisateur et communique avec le backend via des requêtes HTTP/HTTPS (API REST).

Application mobile Flutter

L'application mobile, développée avec **Flutter**, s'exécute sur les appareils Android des utilisateurs. Elle communique avec le même backend Laravel que l'application web, via des appels API REST, garantissant une cohérence des données entre les deux plateformes.

Serveur Laravel

Le backend **Laravel** héberge la logique métier de l'application : authentification (Breeze), gestion des rôles (Spatie), gestion des consultations, conversations et documents. Il expose une API REST consommée à la fois par le client web et l'application mobile.

Base de données

La base de données **MySQL** stocke l'ensemble des données persistantes de l'application : utilisateurs, rôles, consultations, conversations, messages et métadonnées des documents réglementaires.

Service IA ou API IA

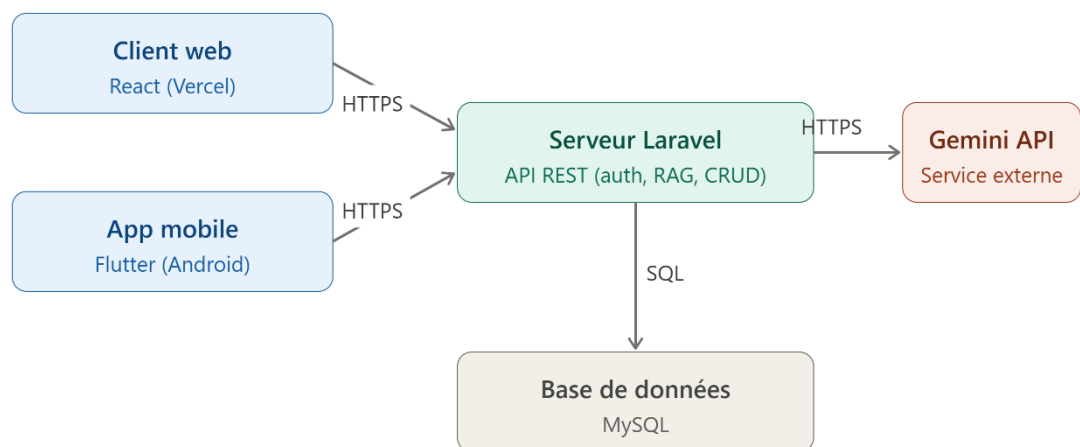
Le service **Gemini API** (Google) est un service externe consommé par le backend Laravel. Il reçoit les questions des utilisateurs accompagnées des documents réglementaires pertinents, et retourne des réponses contextualisées générées par intelligence artificielle.

Communication entre les composants



Composant source	Composant cible	Protocole
Client web (React)	Serveur Laravel	HTTPS / API REST (JSON)
Application mobile (Flutter)	Serveur Laravel	HTTPS / API REST (JSON)
Serveur Laravel	Base de données MySQL	Connexion SQL (Eloquent ORM)
Serveur Laravel	Service Gemini API	HTTPS / API REST (requêtes avec documents PDF)

Le schéma se représente ainsi visuellement dans PowerDesigner :



Chapitre 4 : Architecture technique et technologies utilisées

4.1 Architecture générale de l'application

Architecture web

L'application web de Valiquo repose sur une **architecture découplée** (decoupled architecture), où le frontend et le backend sont entièrement séparés et communiquent exclusivement via une API REST. Le frontend, développé en **React (TypeScript)**, est responsable de l'affichage et de l'interaction utilisateur, tandis que le backend, développé en **Laravel**, gère la logique métier, l'authentification, la persistance des données et la communication avec le service IA externe.

Architecture mobile

L'application mobile, développée en **Flutter**, suit le même principe d'architecture découplée. Elle consomme la même API Laravel que l'application web, garantissant ainsi une cohérence totale des données entre les deux plateformes sans duplication de logique métier côté serveur.

Communication entre frontend, backend, base de données et service IA

La communication entre les différentes couches du système s'effectue comme suit :

1. Le **frontend** (React ou Flutter) envoie des requêtes HTTP au **backend Laravel** via une API REST au format JSON
2. Le **backend Laravel** traite la requête, interagit avec la **base de données MySQL** via l'ORM Eloquent
3. Lorsque la requête concerne une question réglementaire, le **backend Laravel** transmet la demande au **service Gemini API**, accompagnée des documents réglementaires pertinents
4. Le **service Gemini API** retourne une réponse générée, que le backend enregistre puis retourne au frontend

Cette architecture en couches garantit une séparation claire des responsabilités, facilitant la maintenance et l'évolution future de la plateforme.

4.2 Technologies côté client

React

Le frontend de Valiquo a été développé avec **React** combiné à **TypeScript**, offrant un typage statique qui réduit les erreurs lors du développement et améliore la maintenabilité du code. React permet de construire une interface utilisateur réactive et modulaire, organisée en composants réutilisables (formulaires, cartes, sidebar, widgets de chat).

TailwindCSS

Le style de l'application est géré via **TailwindCSS**, un framework CSS utility-first qui permet de styliser rapidement les composants directement dans le JSX, sans nécessiter de fichiers



CSS séparés volumineux. Ce choix a permis de maintenir une charte graphique cohérente (palette sombre avec accents turquoise) tout au long du développement.

Vite

L'outil de build utilisé est **Vite**, qui offre un temps de démarrage et de rechargement à chaud (hot reload) particulièrement rapide par rapport aux outils traditionnels comme Webpack, accélérant ainsi le cycle de développement.

React Router

La navigation entre les différentes pages de l'application (landing, scanner, dashboard, coach IA) est gérée par **React Router**, permettant une expérience de navigation fluide sans rechargement complet de la page (Single Page Application).

4.3 Technologies côté serveur

PHP

Le backend de Valiquo repose sur le langage **PHP**, choisi pour sa robustesse, sa large adoption dans le développement web, et sa compatibilité native avec le framework Laravel.

Laravel

Laravel a été retenu comme framework backend pour sa structure MVC claire, sa richesse fonctionnelle (ORM Eloquent, système de migrations, validation intégrée) et son écosystème de packages facilitant l'implémentation de fonctionnalités complexes telles que l'authentification et la gestion des rôles.

Routes

Les routes de l'application sont définies dans le fichier `routes/api.php`, exposant les différents endpoints consommés par le frontend React et l'application mobile Flutter par exemple `/api/login`, `/api/consultations`, `/api/documents`

Contrôleurs

Les contrôleurs Laravel centralisent la logique de traitement des requêtes. Par exemple, le `ConsultationController` gère la réception d'une question réglementaire, la transmission au service Gemini API, et l'enregistrement de la réponse générée.

Modèles

Les modèles Eloquent (`User`, `Consultation`, `Conversation`, `Message`, `Document`) représentent les entités de la base de données et permettent une manipulation orientée objet des données, sans écrire de requêtes SQL manuelles.

Validation



Laravel offre un système de validation intégré, utilisé pour vérifier la conformité des données envoyées par les utilisateurs avant tout traitement — par exemple s'assurer qu'une question réglementaire soumise contient un nombre de caractères suffisant, ou qu'un email d'inscription est valide et unique.

Authentification

L'authentification est gérée via **Laravel Breeze en mode API**, combiné au package **Spatie Laravel Permission** pour la gestion des rôles (entrepreneur / administrateur), comme détaillé au chapitre 5.

4.4 Base de données

SGBD utilisé

La base de données utilisée pour Valiquo est **MySQL**, un système de gestion de base de données relationnel largement répandu, offrant une bonne compatibilité avec Laravel et un support natif via l'ORM Eloquent.

Organisation des tables

La base de données est organisée autour des tables suivantes, conformément au schéma relationnel défini au chapitre 3 :

- **users** : informations des utilisateurs
- **roles et model_has_roles** : gestion des rôles via Spatie Permission
- **consultations** : questions réglementaires et réponses générées
- **conversations et messages** : historique des échanges avec le Coach IA
- **documents** : métadonnées des documents PDF alimentant le système RAG
- **Relations**

Les relations entre les tables suivent la structure établie au chapitre 3 : un utilisateur peut avoir plusieurs consultations, plusieurs conversations, et un administrateur peut gérer plusieurs documents. La relation entre utilisateurs et rôles est gérée via une table pivot (**model_has_roles**), permettant une gestion flexible et évolutive des permissions.

Migrations Laravel

L'ensemble de la structure de la base de données est défini via des **migrations Laravel**, permettant de versionner les modifications de schéma et de garantir une reproductibilité de la base de données sur différents environnements (développement, production). Chaque table dispose de sa propre migration, facilitant la maintenance et l'évolution future du schéma notamment pour l'ajout des futurs modules (V2, V3, V4).

4.5 Application mobile Flutter

Présentation de Flutter

Flutter est un framework open-source développé par Google, permettant de créer des applications mobiles natives à partir d'une base de code unique écrite en langage **Dart**. Ce

choix permet de développer rapidement une application fonctionnelle sur Android, avec une interface fluide et des performances proches du natif.

Rôle de l'application mobile

L'application mobile Flutter de Valiquo offre un accès simplifié et mobile aux fonctionnalités essentielles de la plateforme : authentification, soumission de questions réglementaires, consultation de l'historique, et interaction avec le Coach IA. Elle constitue un point d'accès complémentaire à l'application web, destiné aux entrepreneurs souhaitant consulter rapidement des informations réglementaires depuis leur smartphone.

Communication avec le backend

L'application Flutter communique avec le **backend Laravel** via des requêtes HTTP REST, au format JSON, en utilisant le package `http` de Dart. Les mêmes endpoints API que ceux utilisés par le frontend web sont consommés, garantissant une cohérence totale des données entre les deux plateformes, sans nécessiter de logique métier dupliquée côté serveur.

4.6 Intégration de l'intelligence artificielle

Type d'intégration choisi

L'intégration de l'intelligence artificielle dans Valiquo repose sur la **consommation d'un web service IA externe**, en l'occurrence l'**API Google Gemini**. Ce choix se distingue d'un chatbot entièrement développé en interne ou d'un agent IA autonome : il s'agit ici d'exploiter un service IA déjà existant et performant, auquel sont transmis les documents réglementaires officiels nécessaires à la génération de réponses contextualisées, selon le principe du **RAG (Retrieval-Augmented Generation)**.

Rôle de l'IA dans le projet

L'intelligence artificielle constitue le **cœur de la valeur ajoutée** de Valiquo. Elle permet de transformer des documents juridiques et réglementaires complexes souvent rédigés dans un langage technique difficile d'accès en réponses claires, structurées et contextualisées, adaptées à la question précise posée par l'entrepreneur. Sans cette intégration IA, Valiquo se limiterait à une simple base documentaire statique, sans réelle capacité d'analyse ni de personnalisation des réponses.

Concrètement, le service Gemini API reçoit deux éléments en entrée : la question de l'utilisateur, et les documents PDF réglementaires pertinents (CGI, loi sur la SARL, statut auto-entrepreneur, etc.). Il analyse ces documents et génère une réponse précise, citant si possible les textes ou procédures applicables, avant de la retourner au backend Laravel pour enregistrement et affichage.

4.7 Outils de développement et DevOps

Visual Studio Code

Le développement de l'ensemble du projet frontend, backend et application mobile a été réalisé avec **Visual Studio Code**, assisté de l'extension **Cursor**, qui intègre des capacités

d'intelligence artificielle facilitant la génération, la correction et l'optimisation du code tout au long du projet.

Git / GitHub

Le versionnement du code source a été assuré via **Git**, avec un dépôt hébergé sur **GitHub**. Cela a permis de suivre l'évolution du projet, de conserver un historique des modifications, et de faciliter la restructuration du dépôt lors du changement de frontend en cours de développement.

Postman

L'outil **Postman** a été utilisé pour tester les endpoints de l'API Laravel de manière indépendante du frontend, permettant de valider le comportement du backend (authentification, soumission de questions, gestion des documents) avant son intégration complète avec l'interface utilisateur.

Serveur local

Le développement backend a été réalisé en local via le serveur de développement intégré de Laravel (php artisan serve), couplé à une base de données MySQL locale, avant tout déploiement.

Hébergement et déploiement

Le frontend de l'application est déployé sur **Vercel**, une plateforme d'hébergement adaptée aux applications React/Vite, offrant un déploiement continu à chaque mise à jour du dépôt GitHub.

Documentation README

Un fichier **README** a été rédigé à la racine du projet, présentant les instructions d'installation, les prérequis techniques, et la structure générale du dépôt, conformément aux exigences de livraison du projet.

Chapitre 5 : Réalisation de l'application web

5.1 Présentation générale de l'application web

Objectif de la partie web C'est quoi Valiquo concrètement ? Une plateforme SaaS web accessible depuis un navigateur, qui permet à un entrepreneur marocain de soumettre une question ou un projet entrepreneurial et obtenir une réponse précise basée sur le cadre réglementaire officiel marocain, grâce à un système RAG alimenté par des textes juridiques officiels. L'objectif de la partie web c'est de rendre cet outil accessible sans installation, avec une interface intuitive même pour un non-technicien.

Utilisateurs concernés on a deux types d'utilisateurs : l'entrepreneur (utilisateur standard) qui crée un compte, lance des scans, consulte ses résultats et discute avec le Coach IA et l'administrateur qui gère les utilisateurs et les documents injectés dans le RAG.

Fonctionnalités principales Soumettre une question réglementaire → obtenir une réponse contextualisée basée sur les textes officiels marocains, Coach IA (chat), historique des scans, authentification, dashboard personnel.

Valiquo est conçu comme une plateforme évolutive structurée en quatre modules progressifs, suivant le parcours naturel d'un entrepreneur :

V1 : Module Réglementaire *(version présentée dans ce rapport)*

L'entrepreneur commence par comprendre le cadre légal de son projet : création d'entreprise, statuts juridiques (SARL, auto-entrepreneur), fiscalité, procédures OMPIC. C'est la question fondamentale "Est-ce que mon idée est légale et comment je me structure ?"

V2 : Module Financement

Une fois le cadre légal établi, l'entrepreneur cherche à financer son projet : Maroc PME, Innov Invest, CCG, subventions CRI. "Comment je finance mon projet ?"

V3 : Module Sectoriel

L'entrepreneur analyse le potentiel de son marché par secteur restauration, tech, e-commerce, logistique avec des données comportementales réelles. "Quel est le potentiel de mon marché ?"

V4 : Module Études de Marché

Analyse approfondie basée sur les données officielles HCP, Bank Al-Maghrib, statistiques économiques nationales. "Quelles sont les données macro qui impactent mon secteur ?"

L'architecture technique de la V1 est conçue dès le départ pour accueillir ces modules progressivement, sans refonte majeure.



5.2 Authentification et gestion des utilisateurs

Connexion L'utilisateur accède à la page de connexion et saisit son email et son mot de passe. Si les identifiants sont corrects, il est redirigé vers son tableau de bord personnel. Dans le cas contraire, un message d'erreur explicite s'affiche. Pour implémenter cette fonctionnalité, nous utilisons Laravel Breeze en mode API (configure le backend uniquement pour recevoir et renvoyer du **JSON**), qui génère automatiquement les endpoints d'authentification nécessaires sans vues Blade, ce qui est adapté à notre architecture frontend React séparé du backend Laravel.

Inscription L'utilisateur remplit un formulaire comprenant son prénom, nom, email, mot de passe et sa ville. Laravel valide l'ensemble des données côté serveur avant de créer le compte en base de données MySQL. Le mot de passe est systématiquement **haché avec bcrypt** avant d'être enregistré il n'est jamais stocké en clair.

Gestion des rôles Pour gérer les rôles et permissions de manière propre et évolutive, nous utilisons le package **Spatie Laravel Permission**. Deux rôles sont définis dans l'application :

- **Entrepreneur** : accès à son tableau de bord, ses scans, l'historique et le Coach IA
- **Administrateur** : gestion des utilisateurs, upload des documents PDF pour le RAG, consultation des statistiques globales

Spatie permet d'assigner et de vérifier les rôles de façon simple et professionnelle, et rend l'architecture extensible si de nouveaux rôles sont nécessaires dans les versions futures de Valiquo.

Sécurité des accès Plusieurs mesures de sécurité sont mises en place :

- Hachage des mots de passe via bcrypt
- Protection des routes sensibles par middleware d'authentification
- Validation stricte des données en entrée côté serveur pour prévenir les injections
- Séparation claire des accès selon le rôle de l'utilisateur

5.3 Tableau de bord

Présentation du dashboard

Le tableau de bord est l'espace central de l'utilisateur après connexion. Il est composé de deux zones principales : une **barre de navigation latérale fixe** (sidebar) qui permet de naviguer entre les différentes sections de l'application, et une **zone de contenu principale** qui affiche les informations et actions disponibles. L'interface est personnalisée elle affiche le prénom de l'utilisateur connecté avec un message d'accueil, et lui donne accès immédiatement à ses activités récentes.

Statistiques affichées

Dès l'arrivée sur le dashboard, l'entrepreneur visualise en un coup d'œil 4 indicateurs clés liés à son utilisation de la plateforme :

- **Nombre de consultations** réglementaires effectuées
- **Nombre de questions posées** au Coach IA



- **Thématiques consultées** ex: création d'entreprise, fiscalité, statuts juridiques
- **Historique récent** les dernières conversations et consultations

Ces indicateurs permettent à l'entrepreneur de suivre sa progression dans la compréhension du cadre réglementaire marocain.

Menus principaux

La sidebar contient les sections suivantes :

- **Tableau de bord** : vue d'ensemble et statistiques personnelles
- **Mes projets** : liste des projets et questions soumises
- **Coach IA** : interface de chat avec l'assistant spécialisé en réglementation marocaine
- **Analyses** : historique détaillé des consultations
- **Rapports** : consultation et export des réponses générées
- **Déconnexion** : accessible en bas de la sidebar

5.4 Gestion des principales fonctionnalités

Ajout

Entrepreneur : crée un nouveau projet en soumettant une question réglementaire via le scanner, et démarre de nouvelles conversations avec le Coach IA. Chaque soumission est enregistrée automatiquement en base MySQL liée à son compte.

Administrateur : ajoute de nouveaux utilisateurs manuellement si nécessaire, et surtout **uploades les documents PDF officiels** (CGI, loi sur la SARL, procédures OMPIC...) qui alimentent le RAG via l'interface d'administration. C'est l'action la plus critique de son rôle c'est lui qui enrichit la base de connaissances de l'IA.

Modification

Entrepreneur : modifie les informations de son profil prénom, nom, ville, mot de passe.

Administrateur : modifie les informations d'un utilisateur, met à jour les documents PDF du RAG en remplaçant une version obsolète par une version plus récente par exemple mettre à jour le CGI quand une nouvelle version est publiée.

Suppression et archivage

Entrepreneur : supprime une conversation ou un projet de son historique. Les données sont archivées avant suppression définitive.

Administrateur : désactive un compte utilisateur sans le supprimer définitivement. Supprime ou remplace un document PDF du RAG qui n'est plus en vigueur.

Consultation

Entrepreneur : consulte les réponses générées par le RAG sous forme de rapport structuré, et l'historique complet de ses conversations avec le Coach IA organisé par date et projet.



Administrateur : consulte la liste complète des utilisateurs, leurs activités, le nombre de consultations effectuées, et les documents PDF actuellement injectés dans le RAG avec leurs dates d'ajout.

Recherche

Entrepreneur : recherche dans son historique par mot-clé, thématique réglementaire ou date.

Administrateur : recherche un utilisateur par nom ou email, et filtre les documents du RAG par type ou date d'ajout.

Filtrage

Entrepreneur : filtre son historique par thématique, création d'entreprise, fiscalité, statuts juridiques, procédures OMPIC, et par période.

Administrateur : filtre les utilisateurs par rôle, statut (actif/inactif), ou date d'inscription. Filtre les documents du RAG par catégorie réglementaire.

5.5 Présentation des interfaces

Note : Les interfaces présentées ci-dessous représentent l'état actuel de l'application en cours de conception. Certaines modifications seront apportées notamment pour aligner le contenu et les textes avec la V1 spécialisée en réglementation marocaine.

Écran d'inscription

La page d'inscription présente un formulaire centré composé de six champs : prénom, nom, email, mot de passe et ville. Un bouton "Créer mon compte" en turquoise valide la soumission. Un lien "Se connecter" est disponible en bas pour les utilisateurs déjà inscrits.





Créer mon compte

Rejoins les entrepreneurs marocains

Prénom

Youssef

Nom

El Amrani

Email

ton@email.ma

Mot de passe

• • • • •

Ville

Casablanca

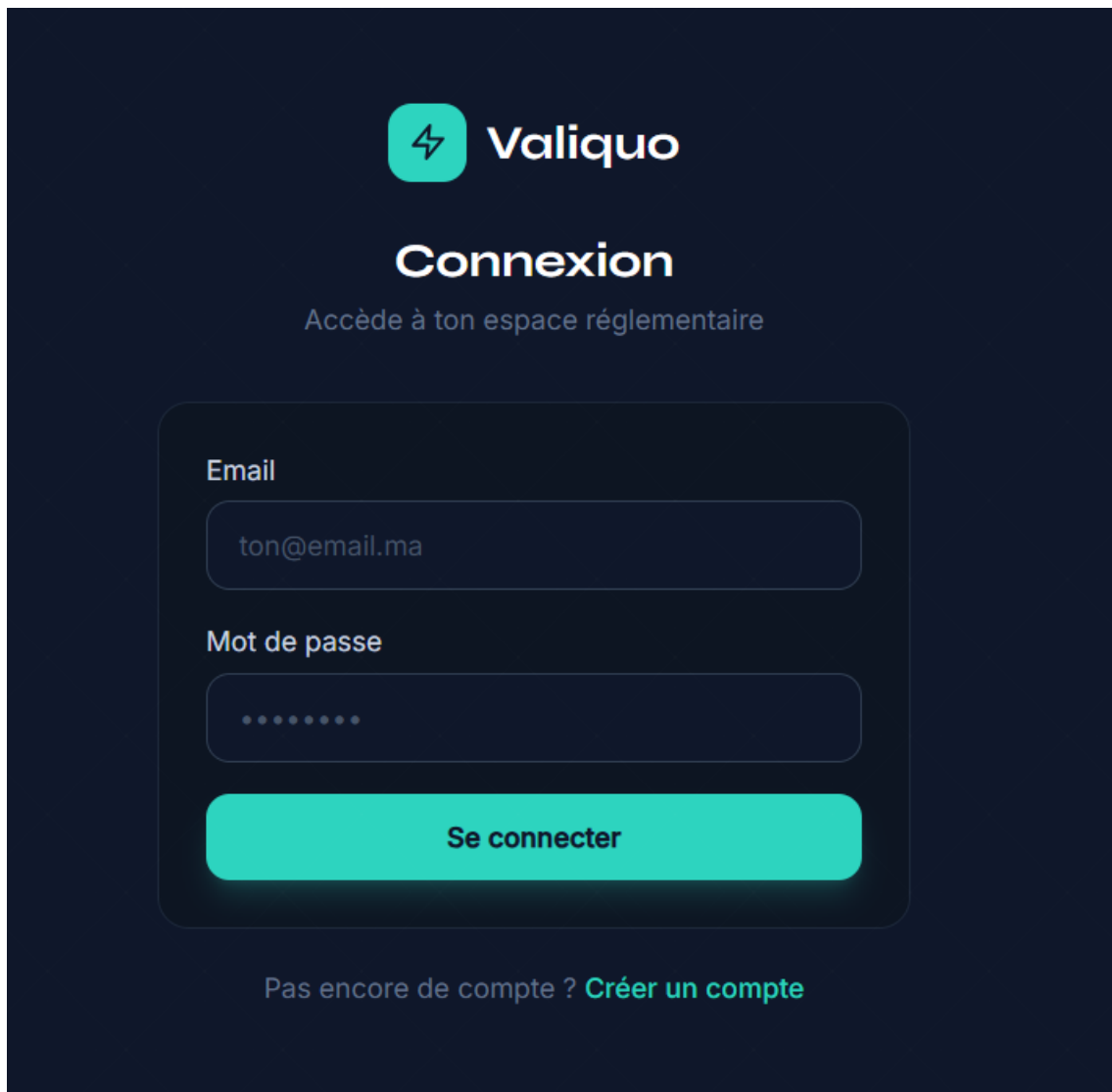
Créer mon compte

Déjà un compte ? [Se connecter](#)

Écran de connexion

La page de connexion affiche un formulaire épuré avec deux champs email et mot de passe et un bouton "Se connecter". Un lien vers la page d'inscription est accessible en bas. Le design respecte la charte graphique de Valiquo fond sombre, accents turquoise.





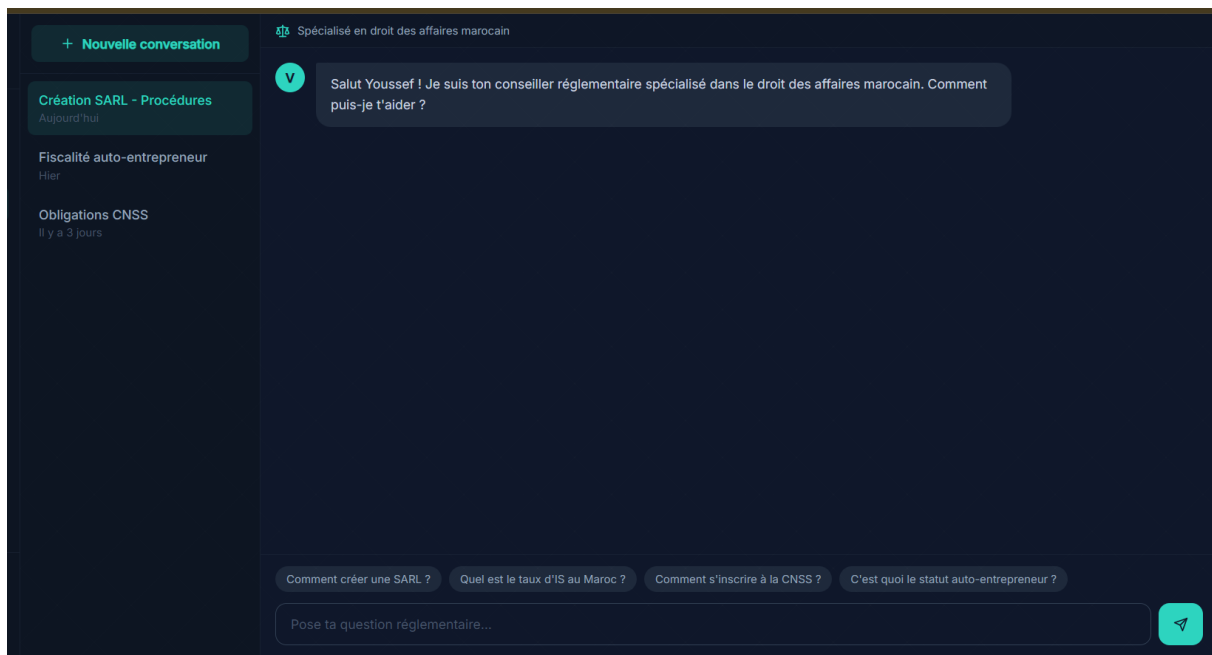
The image shows a login interface for 'Valiquo'. At the top, there is a logo consisting of a teal square with a white lightning bolt icon, followed by the word 'Valiquo' in white. Below the logo, the word 'Connexion' is written in a large, bold, white font. Underneath 'Connexion', the text 'Accède à ton espace réglementaire' is displayed in a smaller, lighter white font. The login form is centered and contains two input fields: 'Email' with the placeholder 'ton@email.ma' and 'Mot de passe' with a series of dots. Below these fields is a large teal button with the text 'Se connecter' in white. At the bottom of the form, there is a link that says 'Pas encore de compte ? Créer un compte'.

Interface Coach IA

L'interface du Coach IA se compose de deux zones : une sidebar gauche listant l'historique des conversations avec un bouton "Nouvelle conversation", et une zone de chat principale affichant les échanges avec l'assistant. Des suggestions de questions apparaissent sous le champ de saisie pour guider l'utilisateur. Le nom de l'utilisateur connecté est visible en bas de la sidebar.

À noter : les suggestions de questions et le contexte du Coach seront mis à jour pour refléter la spécialisation réglementaire ex: "Quelles sont les étapes pour créer une SARL ?", "Quel est le taux d'IS au Maroc ?"





5.6 Difficultés rencontrées et solutions adoptées

Problèmes techniques

1. Séparation frontend / backend Architecturer une application avec un frontend React totalement découplé du backend Laravel a nécessité une réflexion approfondie sur la gestion des requêtes cross-origin (CORS) et sur le choix du système d'authentification adapté. Plusieurs tentatives ont été nécessaires avant de trouver la configuration stable. *Solution adoptée* : Configuration manuelle du middleware CORS dans Laravel et adoption de Laravel Breeze en mode API combiné à Spatie Permission pour gérer l'authentification et les rôles de façon propre.

2. Intégration du RAG La mise en place d'un système RAG a représenté l'un des défis les plus importants du projet. Les premières approches envisagées : scraping de données, construction d'une base vectorielle from scratch, ou encore l'API Assistants d'OpenAI se sont révélées soit trop complexes à fiabiliser dans le délai disponible, soit nécessitant un budget incompatible avec les contraintes du projet.

Solution adoptée : Après plusieurs recherches et tests, le choix s'est porté sur l'**API Google Gemini**, qui permet d'envoyer directement les documents PDF dans les requêtes sans nécessiter de gestion manuelle des embeddings, tout en restant entièrement gratuite dans les limites d'utilisation du projet (1500 requêtes/jour). Cette solution offre un bon compromis entre simplicité d'intégration, fiabilité des réponses et absence de coût.

3. Déploiement continu La gestion du déploiement sur Vercel a nécessité plusieurs ajustements, notamment après la refonte du frontend. Le fichier de configuration pointait vers



des chemins obsolètes, causant des déploiements incorrects. *Solution adoptée* : Restructuration de l'arborescence du projet et mise à jour manuelle du fichier `vercel.json`

Problèmes de conception

1. Définition du périmètre L'une des décisions les plus difficiles a été de restreindre le périmètre de la V1. Le concept initial de Valiquo couvrait l'ensemble des secteurs du marché marocain, ce qui semblait ambitieux mais rendait la démonstration technique moins convaincante. *Solution adoptée* : Après échanges avec les encadrants, le choix a été fait de spécialiser la V1 sur le module réglementaire, tout en conservant la vision globale de la plateforme pour les versions futures.

2. Cohérence frontend / concept Le frontend ayant été développé en parallèle des décisions conceptuelles, certains éléments d'interface ne correspondaient plus à l'orientation finale de la V1. *Solution adoptée* : Identification précise des éléments à ajuster : textes, placeholders, suggestions sans remettre en cause le travail de design déjà réalisé.

Corrections apportées

- Restructuration complète de l'arborescence du projet
- Mise à jour de la configuration de déploiement
- Suppression des dépendances inutilisées au profit d'une architecture plus cohérente
- Recentrage progressif du contenu de l'interface sur la spécialisation réglementaire



Chapitre 6 : Réalisation de l'application mobile Flutter

6.1 Objectif de l'application mobile

Rôle de la partie mobile dans le projet

L'application mobile Flutter de Valiquo a pour objectif d'offrir aux entrepreneurs marocains un accès rapide et simplifié aux fonctionnalités essentielles de la plateforme, directement depuis leur smartphone. Elle complète l'application web en permettant une consultation à tout moment, sans nécessiter l'ouverture d'un navigateur.

Public cible

L'application mobile s'adresse au même public que l'application web les entrepreneurs marocains mais répond plus spécifiquement à un usage de mobilité, lorsque l'utilisateur souhaite poser une question rapide ou consulter son historique en déplacement.

Fonctionnalités mobiles prévues

- Authentification (connexion / inscription)
- Tableau de bord d'accueil avec statistiques personnelles
- Soumission de nouvelles questions réglementaires
- Interface de chat avec le Coach IA, incluant des suggestions de questions
- Consultation et gestion du profil utilisateur
- Navigation simplifiée via une barre de navigation inférieure (Accueil / Coach IA / Profil)

6.2 Interfaces principales

Écran de connexion

L'écran de connexion présente le logo Valiquo, un champ email pré-rempli à titre d'exemple, un champ mot de passe avec option d'affichage, un lien "Mot de passe oublié", le bouton principal "Se connecter" en turquoise, et un lien vers la création de compte.

Écran d'accueil

L'écran d'accueil affiche un message de bienvenue personnalisé ("Bonjour Ahmed"), trois indicateurs statistiques sous forme de cartes (Consultations, Questions IA, Thématiques), ainsi qu'une liste des consultations récentes avec leur thématique et leur date. Un bouton flottant "Nouvelle question" permet d'accéder rapidement au formulaire de soumission.

Liste des données

La liste des consultations récentes est affichée sous forme de cartes cliquables, chacune indiquant le titre de la consultation, sa thématique (avec badge coloré) et sa date de création.



Interface Coach IA

L'écran du Coach IA propose une interface de chat avec un message d'accueil ("Comment puis-je vous aider ?"), des suggestions de questions pré-définies (chips), et un champ de saisie avec bouton d'envoi en turquoise.

Page de profil

La page de profil affiche l'avatar de l'utilisateur (initiale), son nom, son email, sa ville sous forme de badge, ainsi que ses informations personnelles détaillées dans une carte dédiée.

6.3 Communication avec le backend Laravel

API utilisée

L'application mobile Flutter communique avec le **backend Laravel** via les mêmes endpoints API REST que ceux consommés par le frontend web React, garantissant une cohérence totale des données entre les deux plateformes. Les échanges se font au format **JSON** via le protocole **HTTPS**.

Requêtes envoyées

Les principales requêtes envoyées par l'application mobile sont :

- POST /api/login : authentification de l'utilisateur
- POST /api/register : création d'un nouveau compte
- GET /api/consultations : récupération de l'historique des consultations
- POST /api/consultations : soumission d'une nouvelle question réglementaire
- GET /api/conversations : récupération des conversations avec le Coach IA
- POST /api/conversations/{id}/messages : envoi d'un nouveau message au Coach IA
- GET /api/user : récupération des informations du profil utilisateur

Données reçues

En réponse, l'API Laravel retourne des données structurées au format JSON — par exemple, pour une consultation, l'objet retourné contient la question posée, la réponse générée par le service Gemini API, la thématique, la ville et la date de création. Pour le profil utilisateur, les données retournées incluent le nom, l'email, la ville et les statistiques globales (nombre de consultations, nombre de questions au Coach IA).

Affichage des résultats

Les données reçues sont ensuite affichées dans l'interface Flutter à l'aide de widgets adaptés ListView pour l'historique des consultations, Card pour chaque élément, et bulles de chat personnalisées pour les échanges avec le Coach IA. Le package http de Dart est utilisé pour effectuer les requêtes, et les réponses JSON sont décodées via jsonDecode avant d'être converties en objets Dart exploitables par l'interface.



6.4 Captures d'écran de l'application mobile

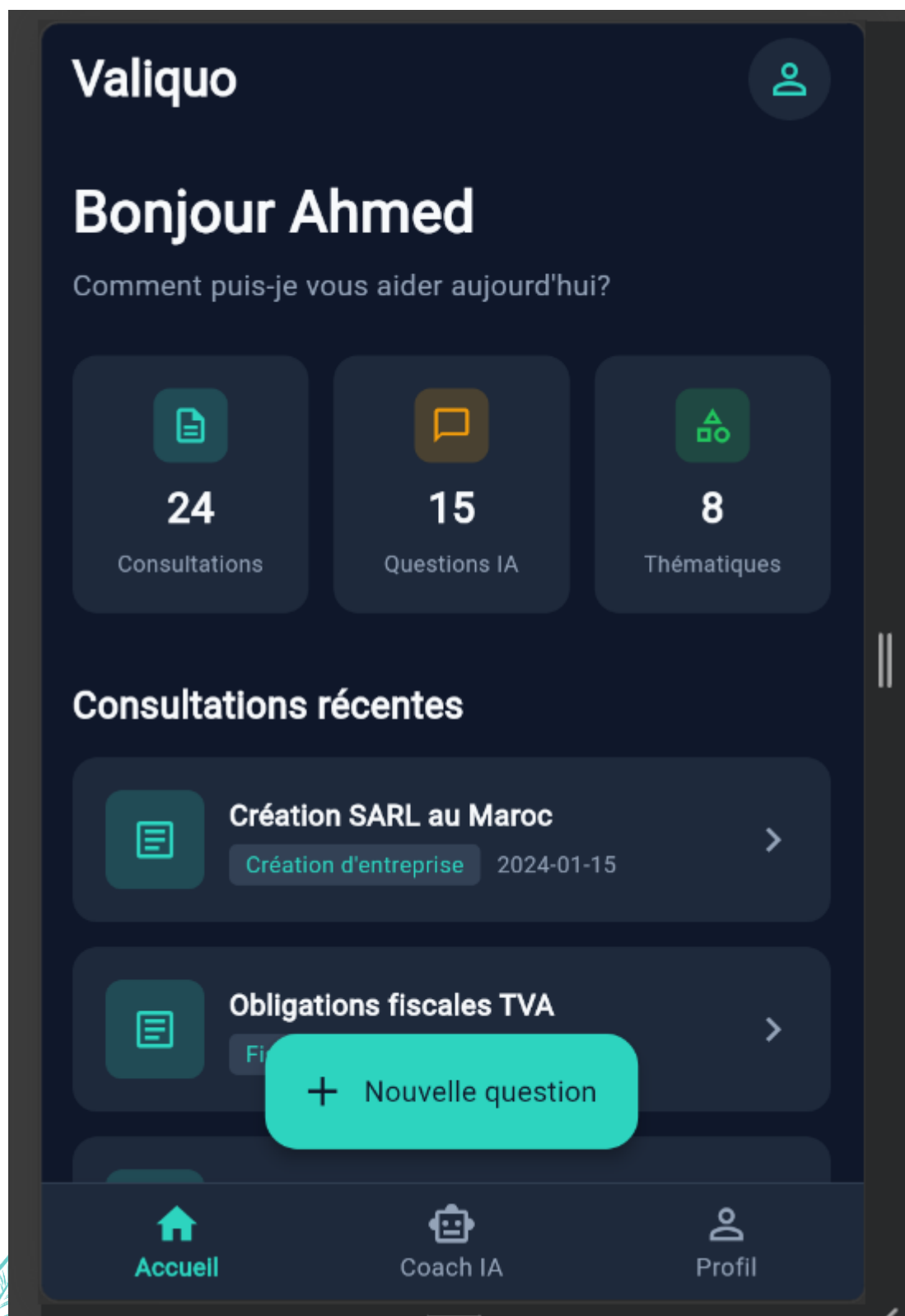
Écran de connexion

L'écran de connexion présente une interface épurée avec le logo Valiquo centré, un champ email, un champ mot de passe avec icône d'affichage/masquage, et le bouton "Se connecter" en turquoise. Le design respecte la charte graphique de l'application web.



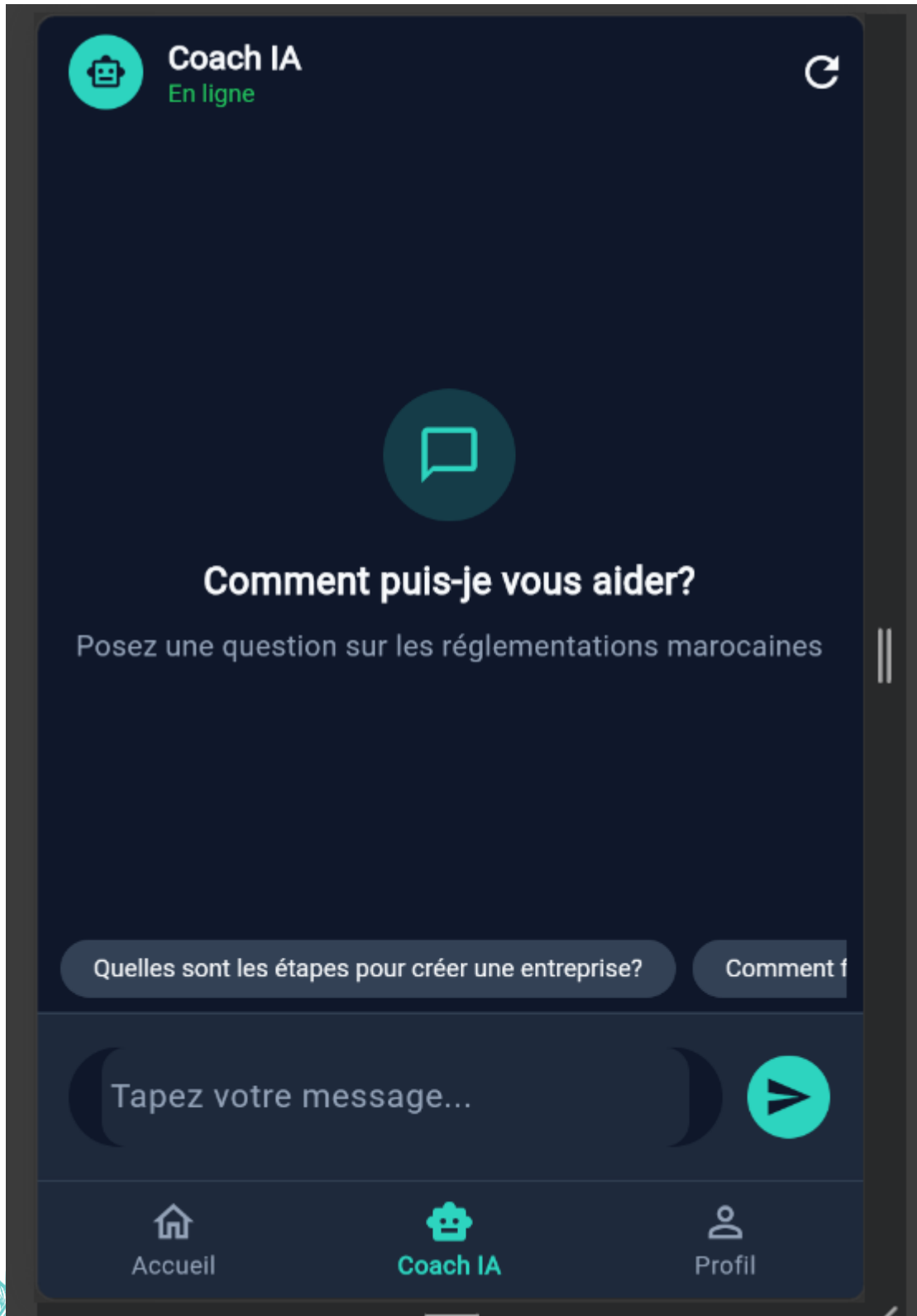
Écran d'accueil

L'écran d'accueil affiche un message de bienvenue personnalisé, trois cartes statistiques (Consultations : 24, Questions IA : 15, Thématiques : 8), et la liste des consultations récentes avec leurs thématiques respectives (ex: "Création SARL au Maroc" Création d'entreprise, "Obligations fiscales TVA" Fiscalité).



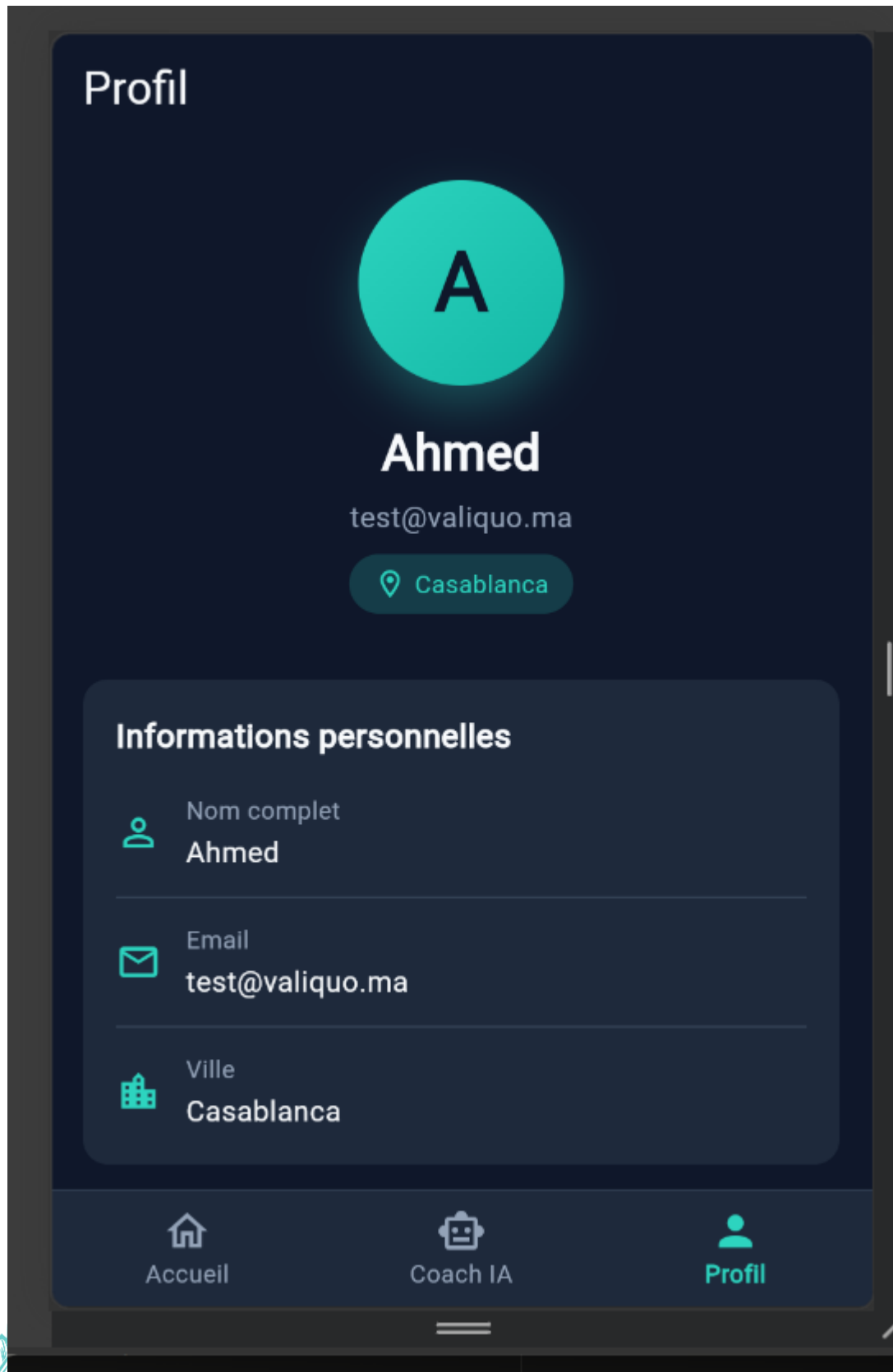
Interface Coach IA

L'écran du Coach IA propose une interface de chat avec indicateur "En ligne", un message d'accueil contextualisé sur les réglementations marocaines, des suggestions de questions rapides ("Quelles sont les étapes pour créer une entreprise ?"), et un champ de saisie avec bouton d'envoi.



Page de profil

La page de profil affiche l'avatar de l'utilisateur, son nom complet, son email et sa ville, ainsi qu'un récapitulatif détaillé de ses informations personnelles dans une carte structurée.



Chapitre 7 : Intégration de l'intelligence artificielle

7.1 Objectif de l'intégration IA

Pourquoi intégrer l'IA ?

L'intégration de l'intelligence artificielle constitue le cœur fonctionnel de Valiquo. Sans elle, la plateforme se limiterait à une base documentaire statique, sans capacité réelle d'analyse ni de personnalisation des réponses selon la situation spécifique de chaque entrepreneur.

Problème résolu par l'IA

L'IA permet de transformer des documents réglementaires officiels souvent longs, denses et rédigés dans un langage juridique complexe en réponses claires, contextualisées et directement exploitables par un entrepreneur non-spécialiste, en fonction de la question précise qu'il pose.

Valeur ajoutée pour l'utilisateur

Grâce à l'IA, l'entrepreneur obtient une réponse personnalisée en quelques minutes, sans avoir à lire l'intégralité des textes de loi ni à consulter un expert payant. La réponse générée s'appuie directement sur les textes officiels en vigueur, garantissant une fiabilité supérieure à une recherche générique sur internet.

7.2 Solution IA choisie

La solution retenue pour Valiquo repose sur la **consommation d'un web service IA externe**, à savoir l'**API Google Gemini**, combinée à une approche de **réponse intelligente basée sur les données** selon le principe du RAG (Retrieval-Augmented Generation).

Contrairement à un chatbot entièrement développé en interne, ou à un agent IA autonome capable de réaliser des actions complexes de manière indépendante, l'approche choisie consiste à transmettre à l'API Gemini, pour chaque question posée, les documents réglementaires officiels pertinents (au format PDF), accompagnés de la question de l'utilisateur. Le modèle Gemini analyse alors ces documents et génère une réponse contextualisée, basée exclusivement sur les sources fournies.

Ce choix a été motivé par plusieurs facteurs :

- **Simplicité d'intégration** l'API Gemini permet d'envoyer directement des fichiers PDF dans la requête, sans nécessiter la construction d'une base vectorielle ou la gestion manuelle d'embeddings
- **Gratuité** l'API Gemini offre un quota gratuit généreux (jusqu'à 1500 requêtes par jour), compatible avec les contraintes budgétaires du projet
- **Fiabilité du contenu** en limitant les sources consultées par l'IA aux seuls documents officiels injectés, les réponses générées restent ancrées dans la réglementation



marocaine réelle, plutôt que dans des connaissances générales potentiellement obsolètes.

7.3 Fonctionnement technique

Question ou demande de l'utilisateur

L'entrepreneur soumet une question réglementaire via le formulaire scanner de l'application web, ou via l'écran "Nouvelle question" de l'application mobile Flutter. La question peut être accompagnée d'informations complémentaires telles que la ville et la thématique concernée.

Envoi de la demande vers Laravel

Le frontend (React ou Flutter) envoie la question saisie au backend Laravel via une requête HTTP POST vers l'endpoint `/api/consultations`, au format JSON.

Traitement par le service IA

Le backend Laravel reçoit la requête, identifie les documents réglementaires pertinents selon la thématique sélectionnée (par exemple les documents liés à la fiscalité si la question concerne le régime fiscal), puis transmet à l'**API Gemini** la question de l'utilisateur accompagnée des documents PDF correspondants. Le modèle Gemini analyse ces éléments et génère une réponse structurée.

Réception de la réponse

Le backend Laravel reçoit la réponse générée par l'API Gemini sous forme de texte structuré, qu'il traite avant de la retourner au frontend.

Affichage à l'utilisateur

La réponse est affichée à l'utilisateur sous la forme d'un rapport structuré, comprenant les points clés, les références aux textes officiels consultés, et un plan d'action recommandé lorsque cela est pertinent.

Enregistrement dans la base de données

Avant l'affichage final, la question et la réponse générée sont enregistrées dans la table consultations de la base de données MySQL, associées à l'utilisateur connecté, garantissant ainsi la traçabilité et la possibilité de consultation ultérieure via l'historique.



7.4 Exemples d'utilisation

Exemple 1 : question posée par l'utilisateur

Un entrepreneur souhaitant créer une SARL à Casablanca soumet la question suivante via le scanner :

"Je veux créer une SARL à Casablanca avec un associé. Quelles sont les étapes, le capital minimum et les obligations fiscales ?"

Exemple 2 : réponse proposée par l'IA

En s'appuyant sur les documents officiels injectés (Loi 5-96 sur la SARL, Guide OMPIC, CGI 2024), le système génère une réponse structurée présentant :

- Le capital minimum requis (1 MAD depuis la réforme de 2018)
- Le nombre d'associés autorisés (de 1 à 50 personnes)
- Le délai moyen de création via le CRI de Casablanca (environ 72 heures)
- Le coût estimé de la procédure (entre 1 000 et 3 000 MAD)
- Un plan d'action en trois étapes : rédaction des statuts, dépôt au CRI, immatriculation à l'OMPIC

Exemple 3 : orientation vers un service ou une fonctionnalité

À la suite de cette première réponse, l'utilisateur peut poursuivre la conversation avec le **Coach IA** pour approfondir un point spécifique, par exemple en demandant : *"Quelles sont mes obligations envers la CNSS une fois la SARL créée ?"* le Coach IA orientant alors l'utilisateur vers les démarches d'inscription sociale obligatoires.

7.5 Limites et perspectives de l'IA

Limites de la solution actuelle

La solution actuelle présente plusieurs limites qu'il convient de souligner avec honnêteté :

- **Périmètre documentaire restreint** la base de connaissances repose sur un nombre limité de documents officiels (7 documents couvrant la fiscalité, la création d'entreprise et le droit des affaires), ce qui ne permet pas encore de couvrir l'ensemble des situations réglementaires possibles
- **Dépendance à un service externe** la qualité et la disponibilité des réponses dépendent entièrement de l'API Gemini, dont les conditions d'utilisation gratuite (quota journalier) pourraient devenir limitantes en cas d'usage intensif
- **Absence de mise à jour automatique** les documents réglementaires doivent être ajoutés ou actualisés manuellement par l'administrateur ; toute évolution législative nécessite une intervention humaine pour rester à jour
- **Spécialisation limitée à la V1** la solution actuelle se concentre exclusivement sur le module réglementaire, sans couvrir les aspects financement ou analyse sectorielle prévus dans les versions futures

Améliorations possibles

- Élargir la base documentaire à un nombre plus important de textes officiels et de circulaires administratives
- Mettre en place un système d'alerte permettant à l'administrateur d'être notifié lorsqu'un texte de loi est mis à jour
- Améliorer la précision des réponses en affinant le système de sélection des documents transmis à l'IA selon la thématique de la question
- Ajouter une fonctionnalité de citation précise des articles de loi consultés

Évolution future du module IA

À terme, le module IA de Valiquo est conçu pour évoluer vers les versions V2, V3 et V4 présentées au chapitre 5 intégrant respectivement les données de financement, les études sectorielles, et les statistiques macroéconomiques officielles. Cette évolution permettra à l'IA de fournir un accompagnement de plus en plus complet à l'entrepreneur marocain, depuis la création de son entreprise jusqu'à l'analyse approfondie de son marché.



Chapitre 8 : Tests, validation et livraison

8.1 Stratégie de test

La stratégie de test adoptée pour Valiquo vise à vérifier le bon fonctionnement de chaque composant du système, ainsi que la cohérence des échanges entre les différentes couches de l'application (frontend, backend, base de données, service IA).

Tests fonctionnels

Les tests fonctionnels visent à vérifier que chaque fonctionnalité répond aux besoins exprimés au chapitre 2 par exemple, vérifier qu'un utilisateur peut s'inscrire, se connecter, soumettre une question réglementaire, et consulter son historique sans erreur.

Tests d'intégration

Les tests d'intégration permettent de vérifier la bonne communication entre les différentes couches du système notamment l'envoi correct d'une requête depuis le frontend React vers l'API Laravel, puis la transmission de la question au service Gemini API, et enfin l'enregistrement de la réponse dans la base de données MySQL.

Tests de l'API

Les endpoints de l'API Laravel sont testés via **Postman**, en vérifiant notamment :

- Le bon fonctionnement de l'authentification (/api/login, /api/register)
- La création d'une consultation (POST /api/consultations)
- La récupération de l'historique (GET /api/consultations)
- Le comportement attendu en cas de données invalides ou manquantes (réponses d'erreur appropriées)

Tests de l'interface web

L'interface web est testée manuellement à travers les différents parcours utilisateur inscription, connexion, soumission d'une question, consultation du résultat, interaction avec le Coach IA en vérifiant la cohérence de l'affichage et l'absence d'erreurs visuelles ou fonctionnelles.

Tests de l'application mobile

L'application Flutter est testée sur émulateur Android, en vérifiant la navigation entre les écrans, la communication avec l'API Laravel, ainsi que la cohérence des données affichées par rapport à celles visibles sur l'application web.

Tests de la partie IA

Les réponses générées par l'API Gemini sont évaluées qualitativement en soumettant plusieurs questions réglementaires types, et en vérifiant que les réponses obtenues sont cohérentes avec les documents officiels injectés, pertinentes par rapport à la question posée, et correctement structurées.

8.2 Tableau des tests réalisés

Cette section présente les principaux tests réalisés sur les différentes couches du système, conformément à la stratégie de test définie au point 8.1.

Fonctionnalité testée	Scénario de test	Résultat attendu	Résultat obtenu	Observation
Inscription utilisateur	Créer un compte avec email, mot de passe, prénom, nom, ville	Compte créé, rôle "entrepreneur" assigné automatiquement	✓ Conforme	Test automatisé validé
Connexion utilisateur	Se connecter avec des identifiants valides	Redirection vers le tableau de bord, session active	✓ Conforme	Authentification via Sanctum (session/cookie)
Connexion invalide	Se connecter avec un mot de passe incorrect	Message d'erreur, accès refusé	✓ Conforme	Code HTTP 401 retourné
Accès aux données d'un autre utilisateur	Tenter de consulter une consultation appartenant à un autre utilisateur	Accès refusé	✓ Conforme	Code HTTP 403 retourné (policy Laravel)
Création d'une consultation	Soumettre une question réglementaire avec thématique et ville	Consultation enregistrée en base, statut "en_cours"	✓ Conforme	20/20 tests de persistance validés
Consultation de l'historique	Récupérer la liste des consultations de l'utilisateur connecté	Liste paginée, triée par date décroissante	✓ Conforme	Pagination 10 éléments/page fonctionnelle
Création d'une conversation Coach IA	Démarrer une nouvelle conversation et envoyer un message	Message enregistré, réponse du coach générée et sauvegardée	✓ Conforme	26/26 tests d'intégration validés (HTTP simulé)
Extraction de texte des documents PDF	Extraire le texte d'un document réglementaire officiel	Texte extrait et mis en cache correctement	✓ Conforme	17/17 tests unitaires validés
Sélection du document pertinent	Associer une thématique au document	Document correct sélectionné selon la thématique	✓ Conforme	Mapping thématique → PDF fonctionnel

Fonctionnalité testée	Scénario de test	Résultat attendu	Résultat obtenu	Observation
	réglementaire correspondant			
Appel réel à l'API Gemini	Envoyer une question réelle au service Gemini avec le document associé	Réponse générée par l'IA basée sur le document	⚠️ ☐ Partiellement conforme	Erreur HTTP 429 (quota gratuit insuffisant sur la clé API utilisée) — l'architecture et le flux technique sont fonctionnels et validés via tests simulés (mocks HTTP)
Validation des données d'entrée	Soumettre un formulaire avec des champs invalides ou manquants	Message d'erreur de validation explicite	✓ Conforme	Code HTTP 422 retourné

8.3 Validation par rapport aux besoins

Comparaison avec les besoins fonctionnels

Cette section compare les besoins fonctionnels identifiés au chapitre 2 avec leur niveau de réalisation effective dans le projet.

Besoin fonctionnel	Niveau de réalisation
Authentification (inscription, connexion, déconnexion)	✓ Réalisé
Gestion des rôles (entrepreneur / administrateur)	✓ Réalisé
Soumission de questions réglementaires	✓ Réalisé
Persistance des consultations en base de données	✓ Réalisé
Génération de réponses contextualisées via l'IA	⚠️ ☐ Partiellement réalisé
Interface de chat avec le Coach IA	✓ Réalisé (architecture et persistance)
Consultation de l'historique	✓ Réalisé
Gestion des documents réglementaires par l'administrateur	■ Non réalisé
Application mobile Flutter	⚠️ ☐ Partiellement réalisé (interface fonctionnelle, non connectée à l'API)



Comparaison avec les besoins non fonctionnels

Besoin non fonctionnel	Niveau de réalisation
Sécurité (hachage des mots de passe, contrôle d'accès)	✓ Réalisé
Validation des données saisies	✓ Réalisé
Interface responsive	✓ Réalisé
Compatibilité web et mobile	✓ Réalisé (interfaces développées)
Performance des appels API	✓ Réalisé pour les endpoints internes
Fiabilité des réponses IA	⚠ Non testable en conditions réelles à ce stade
Maintenabilité (architecture orientée services)	✓ Réalisé

Fonctionnalités réalisées

- Authentification complète avec gestion des rôles
- Persistance des consultations et conversations en base de données
- Architecture backend orientée services, prête à recevoir l'intégration IA complète
- Sélection automatique des documents réglementaires pertinents selon la thématique
- Extraction et mise en cache du texte des documents PDF
- Interface web complète (landing, scanner, dashboard, Coach IA)
- Interface mobile Flutter fonctionnelle (authentification, dashboard, Coach IA, profil)

Fonctionnalités partiellement réalisées

- **Génération de réponses IA en conditions réelles** : l'architecture technique est entièrement fonctionnelle et validée par des tests automatisés, mais l'appel réel à l'API Gemini est actuellement limité par une contrainte de quota sur la clé API utilisée, indépendante du code développé
- **Connexion de l'application mobile à l'API** : l'interface Flutter est développée mais n'est pas encore branchée sur les véritables endpoints Laravel

Gestion administrative des documents réglementaires : conçue au niveau de la modélisation (chapitre 3) mais non implémentée techniquement à ce stade.

8.4 Livraison du projet

Code source

L'ensemble du code source du projet Valiquo (frontend React, backend Laravel, application mobile Flutter) est organisé et versionné via Git, structuré en dossiers distincts : `project/` pour le frontend web, `backend/valiquo-api/` pour l'API Laravel, et un dossier dédié pour l'application Flutter.

Base de données



La base de données MySQL repose sur un schéma structuré autour des tables `users`, `roles`, `consultations`, `conversations` et `messages`, créées via des migrations Laravel versionnées, garantissant une reproductibilité fiable de l'environnement sur toute nouvelle installation.

Guide d'installation

Un guide d'installation est disponible dans le fichier `README.md` à la racine du projet, détaillant les étapes nécessaires au démarrage du frontend (`npm install` puis `npm run dev`) et du backend (`composer install`, configuration du `.env`, migrations, puis `php artisan serve`).

Guide d'utilisation

L'utilisateur entrepreneur crée un compte, se connecte, puis peut soumettre une question réglementaire via le scanner ou échanger directement avec le Coach IA. L'historique de ses consultations reste accessible depuis son tableau de bord à tout moment.

Lien GitHub

Le code source du projet est disponible à l'adresse suivante :
github.com/AyshaCodes/valiquo-v3-

Présentation de la démonstration

La démonstration du projet lors de la soutenance permettra d'illustrer le parcours complet d'un entrepreneur sur la plateforme : création de compte, authentification sécurisée, soumission d'une question réglementaire, et consultation de l'historique des échanges avec le Coach IA, mettant en évidence l'architecture technique développée (frontend React, backend Laravel orienté services, persistance MySQL, et intégration du service d'intelligence artificielle Gemini).



Conclusion générale

Bilan du projet

Le projet Valiquo a permis de concevoir et de développer une plateforme intelligente destinée à accompagner les entrepreneurs marocains dans la compréhension du cadre réglementaire applicable à leur activité. Initialement envisagé comme un outil de validation d'idées de business à portée généraliste, le projet a évolué, suite aux retours des encadrants pédagogiques, vers une première version spécialisée sur le module réglementaire un recentrage qui a permis de renforcer la pertinence et la faisabilité du projet dans le délai imparti, tout en conservant une vision produit évolutive à travers les modules V2, V3 et V4 envisagés pour les versions futures.

Le projet a couvert l'ensemble des dimensions attendues pour un PFE transversal : conception UML, développement d'une application web (React/Laravel), intégration d'une intelligence artificielle (API Gemini selon le principe du RAG), et développement d'une application mobile complémentaire (Flutter).

Compétences acquises

Ce projet a été l'occasion de développer et de consolider plusieurs compétences techniques et méthodologiques :

- Conception et modélisation UML (cas d'utilisation, classes, séquence, déploiement)
- Développement frontend avec React, TypeScript et TailwindCSS
- Développement backend avec Laravel, incluant l'authentification (Breeze) et la gestion des rôles (Spatie Permission)
- Intégration d'un service d'intelligence artificielle externe selon une approche RAG
- Développement mobile avec Flutter et communication avec une API REST
- Gestion de projet selon un cycle en cascade, incluant l'adaptation du périmètre fonctionnel en cours de réalisation
- Utilisation d'outils de développement assistés par IA (Cursor, Bolt) pour accélérer certaines phases de prototypage

Difficultés rencontrées

Le projet a comporté plusieurs défis, notamment la nécessité de redéfinir le périmètre fonctionnel en cours de développement, la mise en place d'une architecture découplée entre un frontend React et un backend Laravel, ainsi que le choix d'une solution d'intelligence artificielle adaptée aux contraintes budgétaires et temporelles du projet (passage d'une approche envisagée avec OpenAI vers l'API Gemini). Ces difficultés, bien que challengeantes, ont permis de développer une réelle capacité d'adaptation et de prise de décision technique.

Limites du travail

Le travail réalisé présente certaines limites assumées, cohérentes avec le périmètre défini pour cette première version : la base documentaire du système RAG reste restreinte à un nombre limité de textes officiels, l'application mobile Flutter couvre les fonctionnalités essentielles



sans encore intégrer l'ensemble des cas d'usage avancés, et les modules de financement et d'analyse sectorielle, bien que conceptualisés, n'ont pas encore été développés.

Perspectives d'amélioration

Plusieurs perspectives d'évolution se dégagent naturellement de ce travail : l'enrichissement de la base documentaire du module réglementaire, le développement des modules V2 (financement), V3 (sectoriel) et V4 (données macroéconomiques) présentés dans ce rapport, ainsi que l'amélioration continue de la précision des réponses générées par l'intelligence artificielle à mesure que la base de connaissances s'enrichit.

Bibliographie / Webographie

- Documentation officielle Laravel : <https://laravel.com/docs>
- Documentation officielle Flutter : <https://docs.flutter.dev>
- Documentation officielle PHP : <https://www.php.net/docs.php>
- Documentation TailwindCSS : <https://tailwindcss.com/docs>
- Documentation API Google Gemini : <https://ai.google.dev/docs>
- Documentation Spatie Laravel Permission : <https://spatie.be/docs/laravel-permission>
- Code Général des Impôts marocain : tax.gov.ma
- Office Marocain de la Propriété Industrielle et Commerciale (OMPIC) : ompic.ma
- Portail national de l'auto-entrepreneur : autoentrepreneur.ma

